



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Federated Heterogeneous Graph Neural Networks for TBML Detection

A Major Project Report (CS481P)

Submitted by,

Nikhil Vasu

1RV22CS128

Rohith Biradar

1RV22CS164

Suraj Chanaveeragoudra

1RV22CS211

Under the guidance of

Dr. Nagaraj G. S

Prof. Sanjana Ravindra Otihal

Professor

Assistant Professor

Dept. of CSE

Dept. of CSE

RV College of Engineering

RV College of Engineering

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering[®], Bengaluru
(*Autonomous institution affiliated to VTU, Belagavi*)
Department of Computer Science and Engineering



CERTIFICATE

Certified that the major project (CS481P) work titled ***Federated Heterogeneous Graph Neural Networks for TBML Detection*** is carried out by **Nikhil Vasu (1RV22CS128)**, **Rohith Biradar (1RV22CS164)** and **Suraj Chanaveeragoudra (1RV22CS211)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide

Head of the Department

Principal

Dr. Nagaraj G. S

Dr. Shanta Rangaswamy

Dr. K. N. Subramanya

External Viva

Name of Examiners

Signature with Date

1.

2.

DECLARATION

We, **Nikhil Vasu, Rohith Biradar** and **Suraj Chanaveeragoudra** students of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Federated Heterogeneous Graph Neural Networks for TBML Detection**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name

Signature

1. Nikhil Vasu(1RV22CS128)
2. Rohith Biradar(1RV22CS164)
3. Suraj Chanaveeragoudra(1RV22CS211)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Nagaraj G. S**, Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel members **Dr. Nagaraj G. S**, Professor, Department of CSE, RVCE and **Prof. Sanjana Ravindra Otihal**, Assistant Professor, Department of CSE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Dr. Chethana R Murthy**, Professor, Department of CSE and **Prof. Deepika Dash**, Assistant Professor, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Trade-Based Money Laundering (TBML) has become a major challenge in modern financial systems due to the increasing complexity and scale of global trade transactions. Conventional anti-money laundering systems primarily rely on rule-based monitoring and traditional machine learning techniques, which are often inadequate for identifying hidden transactional dependencies and evolving laundering strategies. These approaches suffer from limitations such as high false positive rates, poor relational learning capability, and reduced scalability in large financial networks. To address these issues, this report presents a Federated Heterogeneous Graph Neural Network (HGNN) framework for real-time TBML detection capable of modeling complex trade relationships while preserving institutional data privacy.

The primary objective of this work is to develop a scalable and privacy-preserving TBML detection architecture using graph-based deep learning methodologies. The proposed framework represents financial ecosystems as heterogeneous graphs consisting of entities such as accounts, importers, exporters, transactions, and trade documents. Graph Attention Network (GAT)-based convolutional layers are employed to learn structural and semantic relationships between interconnected entities for effective anomaly detection. Federated learning mechanisms are incorporated to facilitate collaborative model training across distributed financial institutions without centralized data sharing. The computational framework includes graph construction, feature extraction, node embedding generation, suspicious activity classification, and real-time risk score prediction.

The proposed system is implemented using Python, PyTorch Geometric, FastAPI, Memgraph, PostgreSQL, and React.js. Experimental evaluation is carried out on simulated TBML datasets containing both legitimate and suspicious trade transactions. Performance analysis is conducted using standard classification metrics including accuracy, precision, recall, and F1-score. The experimental results indicate that the proposed framework effectively captures hidden laundering patterns and improves fraud detection capability when compared with conventional machine learning approaches. The architecture further supports scalable deployment, reduced false positive rates, and efficient real-time monitoring of high-volume trade transactions.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction to Trade-Based Money Laundering

CHAPTER 1

INTRODUCTION TO TRADE-BASED MONEY LAUNDERING

International trade and digital finance have made global transactions faster, but also harder to monitor. Criminals increasingly exploit this complexity to move illicit value through legitimate trade activity, exposing the limits of conventional Anti-Money Laundering (AML) systems. Trade-Based Money Laundering (TBML) is especially difficult to detect because the suspicious activity is hidden inside normal commercial exchanges. This creates a clear need for intelligent, scalable, and privacy-preserving methods that can uncover hidden transactional relationships in real time.

“For financial institutions, TBML is uniquely challenging because illicit funds are embedded within legitimate trade transactions, leaving traditional AML controls blind to the risks.”

– Dr. Sebastian Hetzler, IMTF Co-CEO [1]

TBML works by blending illicit financial flows into ordinary trade transactions, which reduces the effectiveness of rule-based compliance checks and isolated transaction monitoring.

1.1 Overview of Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) refers to disguising the proceeds of crime through legitimate international trade transactions [2]. Instead of moving money directly, TBML hides illegal value inside import-export activities, invoice settlements, and cross-border trade records. Because these transactions involve multiple intermediaries, institutions, and jurisdictions, conventional AML systems struggle to identify suspicious patterns reliably.

1.1.1 Global Financial Crime Scenario

The growth of cross-border trade has made illicit activity harder to spot in normal financial flows. The United Nations Office on Drugs and Crime (UNODC) estimates that money laundering accounts for nearly 2% to 5% of global GDP, or about \$800 billion to \$2 trillion annually [3]. Europol also reports that authorities intercept less than 1% of

illicit financial flows [4]. These figures show how quickly laundering networks are evolving and why traditional monitoring tools are no longer enough.

As goods, services, and financial assets move across borders, criminal actors can hide suspicious transactions inside legitimate trade activity. Even as banks strengthen Know-Your-Customer (KYC) controls and transaction surveillance, TBML continues to exploit gaps in conventional monitoring.

Table 1.1: Key Statistics Related to Global Trade-Based Financial Crime

Financial Crime Metric	Estimated Value
Global GDP associated with money laundering annually	2% – 5%
Estimated annual TBML economic volume	\$1.6T – \$2.0T
Illicit flows linked to trade misinvoicing in developing economies	87%
Global interception rate of illicit financial flows	<1%
Reduction in false positive alerts using AI-driven monitoring systems	Up to 40%

1.1.2 Trade-Based Money Laundering

TBML allows criminal organizations to move illicit value through trade while avoiding direct cash movement. Common techniques include over-invoicing, under-invoicing, multiple invoicing, false goods descriptions, phantom shipments, and shell entities that distort trade records. Because these actions are spread across real trade operations, they are difficult for regulators and financial institutions to detect.

Global Financial Integrity reports that trade misinvoicing and related TBML activity account for nearly 87% of illicit financial flows from developing economies [5]. Industry estimates also suggest that about \$1.6 trillion to \$2 trillion in illicit value is hidden each year within trade invoices and commercial documents [1]. Together, these findings show how heavily organized crime depends on the international trade system.

1.1.3 Problem Area

Conventional AML systems usually depend on rules, fixed thresholds, and manual review. While these methods can detect isolated anomalies, they often miss the connected relationships that define coordinated laundering activity. TBML networks bring together

importers, exporters, shell companies, offshore entities, logistics providers, and financial intermediaries across multiple jurisdictions, which makes detection even harder.

Traditional machine learning methods also fall short because they treat transactions as independent events. As a result, they miss structural relationships, produce many false positives, and slow down investigations. A further challenge is privacy: institutions cannot freely share sensitive transactional data, which makes centralized detection impractical. These gaps make a strong case for scalable, privacy-preserving analytics that can model distributed financial ecosystems.

1.2 Literature Review

The growth of global finance has led to a large body of AML and TBML research. Earlier work focused on rule-based monitoring, but newer studies increasingly use machine learning, graph analytics, and distributed learning to detect hidden patterns in financial data. This shift is important because TBML does not usually appear as a single suspicious transfer; instead, it is spread across invoices, counterparties, shipping records, account movements, and commercial relationships. For that reason, recent literature has moved from isolated transaction screening toward methods that can model interactions between entities and uncover structural patterns in the wider financial network.

Jiani Fan, Lwin Khin Shar, Ruichen Zhang, Ziyao Liu, Wenzhuo Yang, Dusit Niyato, and Kwok-Yan Lam [6] surveyed deep learning methods for AML and showed that CNNs, RNNs, and GNNs can improve anomaly detection. Their survey is useful because it presents how learning-based techniques can extract richer patterns than fixed rules alone, especially in environments with large transaction volumes. At the same time, the work remained centered on centralized data processing, which makes it less suitable for institutions that cannot share sensitive records directly. This limitation is especially relevant in TBML scenarios, where data from multiple organizations may be needed to understand the full laundering chain.

Rasmus Ingemann Tuffveson Jensen and Alexandros Iosifidis [7] evaluated standard machine learning models for financial anomaly detection, including Logistic Regression, Random Forests, and Gradient Boosting methods. Their results showed that traditional supervised models can outperform static rule-based approaches when the data is well prepared and labeled. However, the study still treated transactions largely as independent events. In practice, laundering activity is often distributed across connected accounts,

counterparties, and repeated trade interactions, so a purely transaction-level view can miss the broader relational structure.

Jie Song, Sijia Zhang, Junghoon Park, Yu Gu, and Ge Yu [8] proposed a dimension expansion framework for learning hidden money laundering activities within transaction networks. The method strengthened transaction representation by combining neighboring account information with contextual interaction features, which is a useful step toward graph-aware fraud detection. The study showed that incorporating relational information can improve the detection of concealed laundering behavior across financial networks. Even so, the approach still relied on manually engineered features, which can reduce flexibility when the structure of the network changes or when the system must operate across distributed institutions.

In another study, Jie Song, Sijia Zhang, Pengyi Zhang, Junghoon Park, Yu Gu, and Ge Yu [9] introduced a Social Attention Graph Neural Network framework for detecting illicit accounts in blockchain transaction systems. Their work demonstrated the value of attention mechanisms in highlighting the most informative neighbors during graph learning. It also reinforced the idea that graph neural networks can outperform traditional machine learning models in suspicious-account detection. However, the study was focused on blockchain transactions rather than trade documents or invoice-based laundering, so it does not fully address the heterogeneous relationships that appear in TBML cases.

Zhong Li, Xueting Yang, and Changjun Jiang [10] proposed a Multi-View Graph-Based Hierarchical Representation Learning framework for organized money laundering group detection. Their use of multiple graph views helped identify coordinated laundering communities and showed that hierarchical graph representations can capture more than simple pairwise links. The work is valuable because it moves beyond flat feature vectors and models fraud as a collaborative activity. Still, the approach depended on predefined graph relations and centralized repositories, which can limit use in privacy-sensitive environments where institutions must keep their own data local.

Abdul Haluk Batur Gezici and Emre Sefer [11] proposed AMLPD, an unsupervised Anti-Money Laundering detection framework based on Personalized PageRank and graph diffusion. This work is important because it reduces dependence on labeled data, which is often expensive and incomplete in real banking systems. The method was able to capture hidden graph structure in sparse financial networks and demonstrated that unsupervised

learning can be practical for anomaly detection. However, the lack of supervised optimization also led to more false positives, which can increase the workload of compliance teams and reduce trust in the system.

Md. Rezaul Karim, Felix Hermesen, Sisay Adugna Chala, Paola De Perthuis, and Avikarsha Mandal [12] investigated scalable semi-supervised graph learning architectures for large-scale financial transaction monitoring. Their evaluation of SkipGCN and EvolveGCN showed that graph learning can scale to bigger transaction datasets while still preserving useful classification performance. This makes the work relevant for real-world fraud monitoring systems that must handle large volumes of records. Even so, the framework assumed centralized graph access and did not directly solve the privacy and regulatory challenges that arise when multiple institutions need to collaborate.

Kyungmo Koo, Minyoung Park, and Byungun Yoon [13] proposed an autoencoder-based suspicious transaction detection framework using Risk-Based Approaches (RBA) for banking systems. Their model reduced detection complexity by learning compact latent representations of transactions and showed that autoencoder-based methods can support internal banking surveillance. The study is useful because it highlights a lighter-weight alternative to more complex graph models. However, it primarily focused on point anomalies at the account level and did not capture the interconnected laundering relationships that are typical in complex trade ecosystems.

Overall, the literature shows a clear shift from rule-based AML toward graph-based and learning-based approaches. Even so, important gaps remain in heterogeneous graph modeling, real-time analysis, privacy-preserving collaboration, and scalability across distributed financial environments. Most existing methods either rely on centralized datasets or focus on banking and cryptocurrency scenarios instead of the trade relationships that define TBML. These gaps motivate an architecture that combines heterogeneous graph learning, federated intelligence, and real-time detection in one framework.

1.3 Motivation

The work, *“Federated Heterogeneous Graph Neural Networks for Real-Time Detection of Trade-Based Money Laundering”*, was selected to address the growing complexity of financial crime in modern trade ecosystems. Conventional AML systems often rely on rule-based checks and isolated transaction analysis, which makes them less effective at identifying coordinated laundering patterns spread across connected financial networks.

In practice, this leads to more false positives, more manual review, and weaker detection of sophisticated TBML activity.

This project addresses those limitations by combining heterogeneous graph learning with federated intelligence to model transaction relationships while preserving institutional privacy. By using graph attention mechanisms and collaborative training across organizations, the framework supports fraud detection without sharing sensitive transactional data directly. It also adds automated Suspicious Transaction Report (STR) generation and dashboard-based monitoring to support real-time compliance analysis.

1.4 Problem Statement

Existing AML systems rely on rules and isolated transaction analysis, which limits their ability to detect TBML and hidden financial networks. Privacy and regulatory constraints also make cross-institution collaboration difficult. A scalable and privacy-preserving framework is therefore needed to model interconnected financial data and detect suspicious activity in real time.

1.5 Objectives

The objectives of the project are

1. To develop a Federated Heterogeneous Graph Neural Network framework for real-time detection of Trade-Based Money Laundering activities.
2. To model complex transactional relationships among financial entities using heterogeneous graph representation and graph attention mechanisms.
3. To generate Suspicious Transaction Reports (STRs) for identified fraudulent transactions to support financial investigation and compliance processes.
4. To design an interactive dashboard for visualization of fraud predictions, transaction analysis, and real-time monitoring of suspicious activities.

1.6 Brief Methodology of the project

The proposed methodology builds a privacy-preserving graph-based framework for real-time TBML detection. Financial transaction data, trade records, customer information, and entity relationships are first collected and cleaned to create structured analysis

datasets. These records are then converted into a heterogeneous graph where accounts, organizations, invoices, and transactions are linked as nodes and edges.

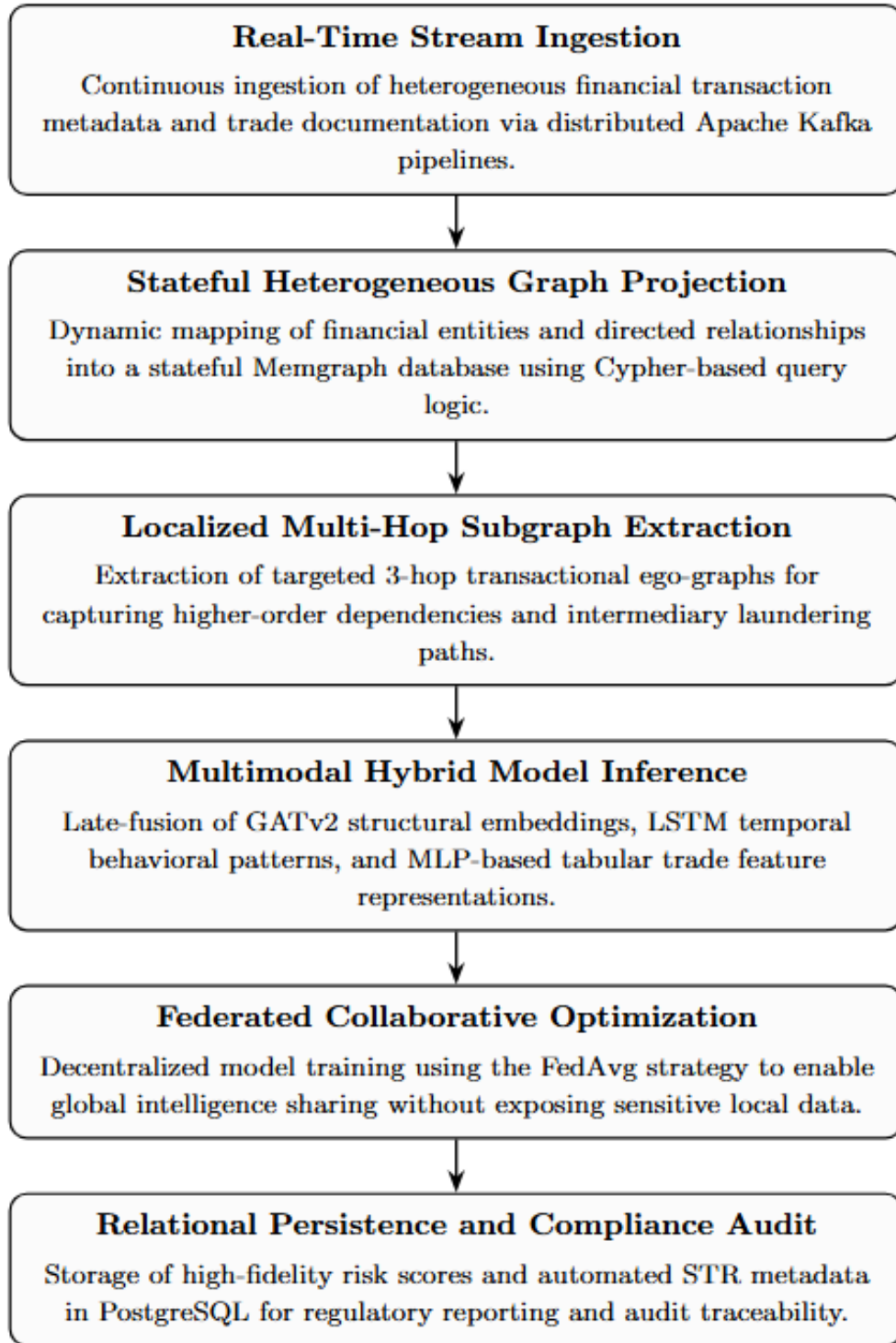


Figure 1.1: Proposed Methodology for TBML Detection

Graph attention mechanisms are used to capture hidden relationships and suspicious behavioral patterns within the financial ecosystem. Federated learning is then added

so that institutions can train collaboratively without exposing sensitive local data. The detected suspicious activities are finally used to generate STRs and support real-time monitoring through an interactive dashboard.

1.7 Assumptions and Constraints of the Work

Assumptions

- The financial transaction records, trade invoices, and customer-related information used for analysis are assumed to be sufficiently structured, consistent, and preprocessed for effective model training and evaluation.
- It is assumed that participating financial institutions maintain similar transactional formats and standardized trade-related attributes required for graph construction and suspicious activity analysis.
- The generated financial datasets are assumed to contain sufficient normal and suspicious transactional patterns necessary for learning hidden behavioral relationships associated with Trade-Based Money Laundering activities.
- It is assumed that heterogeneous graph representations can effectively model relationships among accounts, organizations, transactions, invoices, and trade entities for identifying suspicious financial behavior.

Constraints

- Limited access to real-world Trade-Based Money Laundering datasets due to institutional privacy regulations and confidentiality policies may affect the diversity of transactional data used for evaluation.
- Large-scale graph construction and real-time transaction analysis may increase computational complexity and memory requirements during fraud detection operations.
- The performance of the proposed framework depends significantly on the quality, completeness, and accuracy of the financial data available for model training and testing.
- The proposed framework is validated primarily under controlled academic and experimental conditions and may require additional optimization for large-scale industrial deployment.

1.8 Organization of the Report

This report is organized as follows.

- Chapter 2 discusses the theoretical concepts related to Anti-Money Laundering systems, Trade-Based Money Laundering, Graph Neural Networks, heterogeneous graph learning, and federated learning techniques used in the proposed work.
- Chapter 3 discusses the Software Requirement Specification (SRS) of the proposed system, including functional requirements, performance requirements, software and hardware requirements, and system constraints.
- Chapter 4 discusses the high-level architecture and overall system design of the proposed framework, including workflow diagrams, system components, dashboard architecture, and data flow representation.
- Chapter 5 discusses the detailed design of the proposed system, including graph construction methodology, fraud detection modules, STR generation workflow, and dashboard integration.
- Chapter 6 discusses the implementation details of the proposed framework, including dataset preprocessing, graph generation, backend integration, model development, and dashboard implementation.
- Chapter 7 discusses the testing and validation procedures performed on the proposed system, including functional testing, fraud prediction analysis, dashboard testing, and model evaluation techniques.
- Chapter 8 discusses the experimental results and performance evaluation of the proposed framework using various fraud detection metrics and comparative analysis.
- Chapter 9 discusses the conclusion of the proposed work, existing limitations, and possible future enhancements for improving the framework.

Chapter 2

Theory and Fundamentals of Project

CHAPTER 2

THEORY AND FUNDAMENTALS OF PROJECT

Developing an intelligent framework for Trade-Based Money Laundering detection requires a strong foundation in financial crime analysis, graph theory, and machine learning techniques. This chapter presents the fundamental concepts associated with Trade-Based Money Laundering, Graph Neural Networks, Graph Attention mechanisms, heterogeneous graph representation, and federated learning. The chapter further discusses the computational principles involved in relational transaction modeling, privacy-preserving learning, and real-time fraud analytics. These concepts collectively form the theoretical foundation for the proposed fraud detection framework.

2.1 Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) is a sophisticated form of financial crime in which illicit funds are transferred and concealed through seemingly legitimate international trade activities. Unlike conventional laundering methods that primarily depend on direct financial transactions, TBML exploits import-export systems, trade invoices, shipping documents, and cross-border commercial operations to disguise the movement of illegal funds. Due to the increasing implementation of strict Anti-Money Laundering (AML) regulations and Know Your Customer (KYC) policies, criminal organizations have increasingly shifted toward trade-oriented laundering mechanisms that are comparatively difficult to detect using traditional monitoring systems.

The complexity of modern global trade ecosystems, combined with the involvement of multiple financial entities and transactional relationships, has made TBML detection a major challenge for financial institutions and regulatory agencies. Consequently, intelligent analytical techniques capable of identifying hidden transactional dependencies and suspicious financial behavior have become increasingly important in modern Anti-Money Laundering systems.

2.1.1 Introduction to TBML

Trade-Based Money Laundering primarily involves the manipulation of international trade transactions to illegally transfer value between entities located across different countries. Criminal organizations often misuse trade documentation, invoice values, shipment quantities, and commercial agreements to move illicit funds while making the transac-

tions appear legitimate within global financial systems. Since these laundering activities are embedded within regular import-export operations, identifying suspicious behavior becomes highly challenging for financial institutions and regulatory agencies.

In many TBML operations, colluding buyers and sellers intentionally manipulate the declared value of traded goods to transfer illegal financial assets across borders. Techniques such as over invoicing, under invoicing, phantom shipments, and multiple invoicing are commonly used to conceal the movement of illicit proceeds. Such laundering mechanisms exploit the complexity and high transactional volume of international trade systems, making traditional rule-based Anti-Money Laundering approaches comparatively ineffective in detecting hidden financial relationships.



Figure 2.1: Trade-Based Money Laundering through trade misinvoicing and open-account transactions

Source: Adapted from “The Role of Misinvoicing in the Money Laundering Cycle,” ResearchGate.

Figure ?? illustrates a typical Trade-Based Money Laundering workflow involving collusion between exporters and importers across different jurisdictions.

2.1.2 Common Techniques Used in TBML

Trade-Based Money Laundering activities are generally performed through the manipulation of trade documentation, shipment details, pricing structures, and financial transactions associated with international trade operations. Criminal organizations exploit the complexity of global trade ecosystems to transfer illicit value across borders while making the transactions appear legitimate. Several laundering techniques are commonly used within modern trade systems to disguise the movement of illegal funds and avoid detection from financial monitoring authorities.

Over Invoicing

Over invoicing is a laundering technique in which the value of exported or imported goods is intentionally declared higher than the actual market value. In this process, the buyer transfers an inflated payment amount to the seller, allowing excess funds to be moved across borders under the appearance of legitimate trade transactions. Criminal organizations use this technique to transfer illicit financial assets while avoiding suspicion from conventional banking systems.

Under Invoicing

Under invoicing involves declaring a lower value for traded goods when compared to their actual market price. This technique allows the buyer or seller to secretly transfer additional financial value outside formal banking channels. Under invoicing is commonly used to evade taxes, conceal illegal profits, and facilitate hidden movement of illicit assets across jurisdictions.

Multiple Invoicing

Multiple invoicing refers to the repeated use of the same trade invoice to justify several financial transactions for a single shipment of goods. By duplicating invoices across multiple banking channels or financial institutions, criminal entities can move large amounts of illicit funds while creating the appearance of legitimate trade payments. Detecting such duplicated transactional behavior is challenging in traditional rule-based monitoring systems.

Phantom Shipments

Phantom shipment laundering occurs when financial transactions are conducted for goods that are never actually shipped or delivered. Fraudulent trade documentation and

fabricated shipping records are used to create the illusion of legitimate trade activity while transferring illicit funds between colluding entities. Since no physical goods are exchanged, identifying phantom shipment activities requires strong transactional verification and relational analysis mechanisms.

Misrepresentation of Goods

Misrepresentation of goods involves intentionally providing false descriptions regarding the type, quantity, quality, or category of traded products within commercial documents and invoices. Criminal organizations use this technique to manipulate trade valuations and conceal suspicious financial activities under legitimate business transactions. Such inconsistencies are difficult to identify using isolated transactional analysis methods due to the large volume and diversity of international trade operations.

2.1.3 Challenges in Detecting TBML

Detecting TBML activities is highly challenging due to the complexity and interconnected nature of modern international trade ecosystems. Unlike conventional financial fraud, TBML transactions are often embedded within legitimate commercial operations, making suspicious activities difficult to distinguish from regular trade behavior. Criminal organizations exploit multiple jurisdictions, shell companies, intermediaries, and layered transactional structures to conceal the movement of illicit funds within normal trade systems.

Traditional AML systems primarily depend on rule-based monitoring approaches and isolated transactional analysis techniques. Although these systems are capable of identifying predefined suspicious patterns, they often fail to capture hidden relational dependencies among entities involved in large-scale trade networks. The increasing volume of cross-border financial transactions and evolving laundering strategies further reduce the effectiveness of conventional fraud detection mechanisms.

Another major challenge involves institutional privacy and regulatory compliance constraints associated with financial data sharing. Many organizations are unable to directly exchange sensitive transactional information due to confidentiality policies and legal restrictions. Consequently, modern TBML detection systems require intelligent, scalable, and privacy-preserving analytical frameworks capable of modeling complex financial relationships while supporting real-time fraud detection within large trade ecosystems.

2.2 Algorithms Used

This section presents the major algorithms and deep learning techniques used in the proposed system for traffic prediction and autonomous vehicle navigation. The project combines graph-based learning and sequential modeling approaches to effectively analyze complex traffic patterns and temporal dependencies. Graph Attention Network v2 (GATv2) is utilized to capture spatial relationships between connected road networks through dynamic attention mechanisms, while Long Short-Term Memory (LSTM) is employed to learn temporal traffic variations and long-term sequential dependencies. Together, these algorithms improve traffic prediction accuracy, congestion analysis, and intelligent decision-making within the proposed framework.

2.2.1 Graph Attention Network v2 (GATv2)

Graph Attention Network v2 (GATv2) is a Graph Neural Network (GNN) architecture that uses a dynamic attention mechanism to learn relationships between connected nodes in graph data. It improves feature aggregation by automatically identifying the importance of neighboring nodes. GATv2 is widely used in applications such as traffic prediction, recommendation systems, and autonomous vehicle navigation.

Introduction

Graph Attention Network v2 (GATv2) is an advanced Graph Neural Network (GNN) architecture designed for learning on graph-structured data using a dynamic attention mechanism. It improves upon the original GAT model by overcoming the limitation of static attention and enabling better learning of relationships between connected nodes. GATv2 automatically identifies the importance of neighboring nodes during feature aggregation, making it highly effective for applications such as traffic prediction, recommendation systems, social network analysis, fraud detection, and autonomous vehicle navigation.

Fundamental Concept of GATv2

The primary objective of GATv2 is to generate node embeddings by aggregating information from neighboring nodes using a dynamic attention mechanism. For each node, the model learns which neighboring nodes are important and assigns suitable attention weights before combining their features.

Architecture and Working of GATv2

The overall architecture and workflow of GATv2 are shown in Figure 1.1.

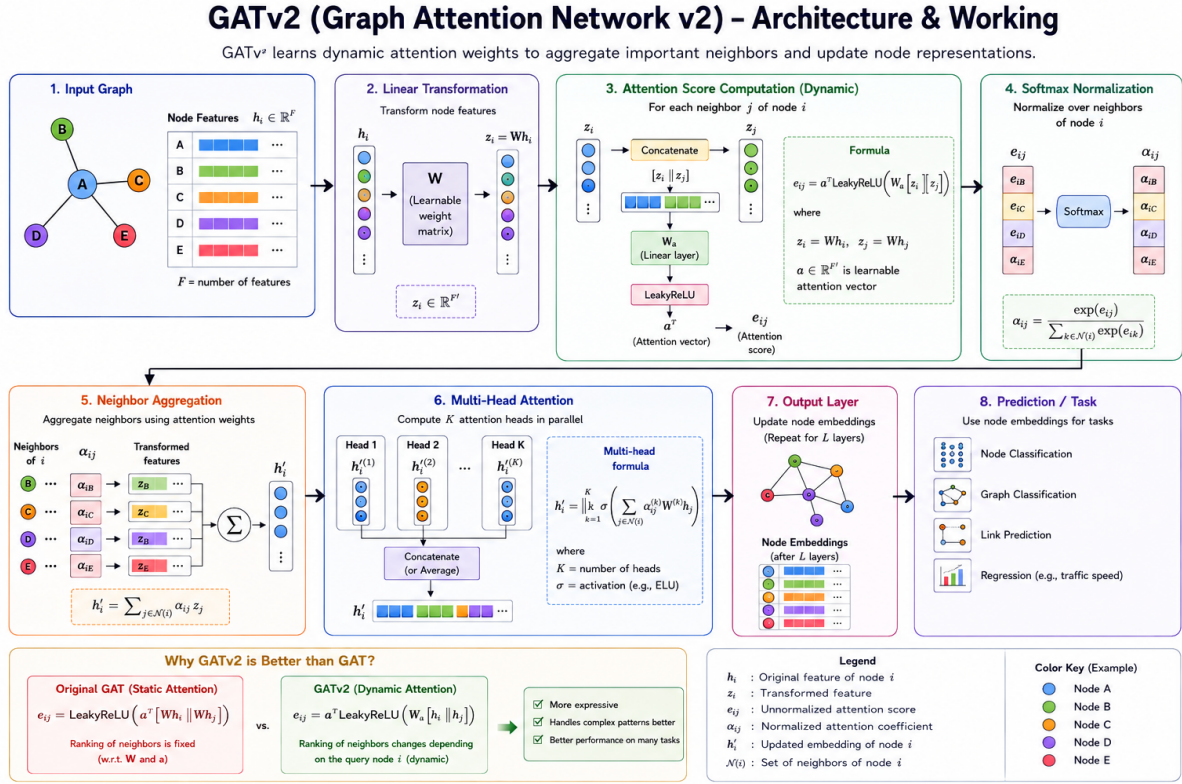


Figure 2.2: GATv2 Architecture and Working

The architecture of GATv2 consists of graph input processing, feature transformation, attention computation, normalization, neighbor aggregation, multi-head attention, and prediction generation.

Step 1: Input Graph: The process begins with a graph containing nodes, edges, and node feature vectors. Each node feature vector is represented as:

$$h_i \in \mathbb{R}^F$$

where h_i denotes the feature vector of node i and F represents the number of input features. In traffic prediction systems, nodes may represent road intersections while edges represent road connectivity.

Step 2: Linear Transformation: The input node features are transformed using a learnable weight matrix:

$$z_i = Wh_i$$

where W is the learnable weight matrix and z_i is the transformed feature representation. This transformation improves feature learning before attention computation.

Step 3: Attention Score Computation: For every connected node pair (i, j) , the attention score is computed as:

$$e_{ij} = a^T \text{LeakyReLU}(W[h_i \| h_j])$$

where a is the learnable attention vector and e_{ij} represents the attention score between neighboring nodes.

Step 4: Softmax Normalization: The attention scores are normalized using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

where α_{ij} denotes the normalized attention coefficient and $\mathcal{N}(i)$ represents the neighboring nodes of node i .

Step 5: Neighbor Aggregation: The neighboring node features are aggregated using weighted summation:

$$h'_i = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} Wh_j$$

where h'_i is the updated node representation.

Step 6: Multi-Head Attention: GATv2 employs multiple attention heads operating in parallel:

$$h'_i = \big\|_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} h_j \right)$$

where K represents the number of attention heads and $\parallel$ denotes concatenation. Multi-head attention improves representation learning and robustness.

Step 7: Output Layer and Prediction: The final node embeddings are passed to the output layer for tasks such as node classification, graph classification, link prediction, and regression.

Advantages of GATv2

GATv2 provides a dynamic attention mechanism that improves feature aggregation and representation learning compared to the original GAT architecture. It automatically identifies important neighboring nodes, performs effectively on irregular graph data, and improves robustness through multi-head attention, making it highly suitable for traffic prediction and autonomous navigation applications.

Limitations of GATv2

Despite its advantages, GATv2 requires high computational and memory resources due to attention computations on large graphs. Training time increases with graph size, and deep GATv2 architectures may suffer from over-smoothing and scalability issues for extremely large datasets.

GATv2 vs GAT

Graph Attention Network v2 (GATv2) was introduced as an improvement over the original Graph Attention Network (GAT) to overcome the limitation of static attention mechanisms. While both architectures use attention-based aggregation for graph representation learning, GATv2 provides a more expressive and adaptive attention mechanism.

In the original GAT architecture, the attention mechanism is defined as:

$$e_{ij} = \text{LeakyReLU} \left(a^T [W h_i \parallel W h_j] \right)$$

In GATv2, the order of operations is modified as:

$$e_{ij} = a^T \text{LeakyReLU} (W [h_i \parallel h_j])$$

This modification allows the attention scores to dynamically depend on both source and neighboring nodes, improving feature interaction learning and representation capability.

Table 2.1: Comparison Between GAT and GATv2

Feature	GAT	GATv2
Attention Type	Static Attention	Dynamic Attention
Neighbor Ranking	Fixed	Context-dependent
Expressiveness	Limited	More expressive
Flexibility	Lower	Higher
Performance	Good	Improved on many graph tasks

GATv2 is preferred over GAT because its dynamic attention mechanism enables better learning of complex graph relationships, resulting in improved feature aggregation, representation learning, and prediction accuracy in graph-based applications.

2.2.2 Long Short-Term Memory (LSTM)

Introduction to LSTM

Long Short-Term Memory (LSTM) is an advanced Recurrent Neural Network (RNN) architecture designed to learn long-term dependencies in sequential data. It was introduced to overcome the vanishing gradient problem faced by traditional RNNs through the use of memory cells and gating mechanisms. LSTM can selectively remember, update, and forget information, making it highly effective for applications such as traffic prediction, speech recognition, natural language processing, and time-series forecasting.

Fundamental Concept of LSTM

The primary objective of LSTM is to preserve important sequential information over long time intervals using memory cells and gating mechanisms. The architecture controls information flow through input, forget, and output gates, enabling efficient learning of temporal dependencies.

Architecture and Working of LSTM

The overall workflow of LSTM consists of gate computation, memory cell update, and hidden state generation.

Input Gate, Forget Gate, and Output Gate

At each time step, the current input and previous hidden state are used to compute the input, forget, and output gates using sigmoid activation functions. These gates regulate information flow into and out of the memory cell.

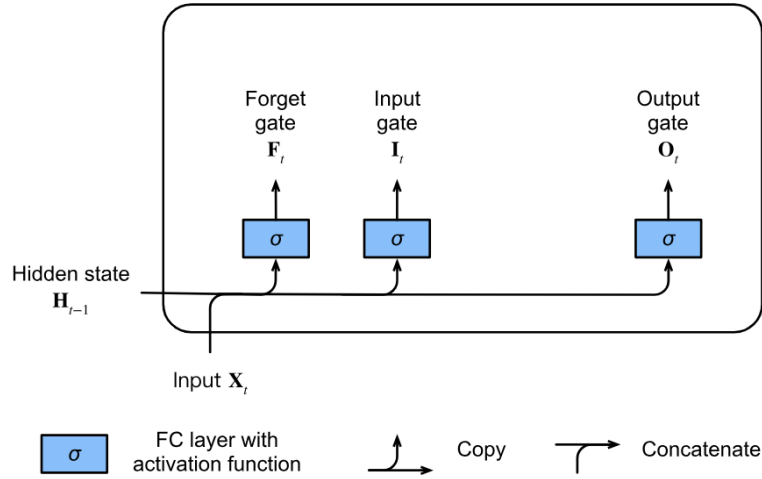


Figure 2.3: Computing the input gate, forget gate, and output gate in an LSTM model

The gates are mathematically represented as:

$$I_t = \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i)$$

$$F_t = \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f)$$

$$O_t = \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o)$$

where I_t , F_t , and O_t represent the input, forget, and output gates respectively, while $\mathbf{X}_t$ and $\mathbf{H}_{t-1}$ denote the current input and previous hidden state.

Input Node

The candidate memory information is computed using the **tanh** activation function as:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

where $\tilde{\mathbf{C}}_t$ represents the candidate memory state.

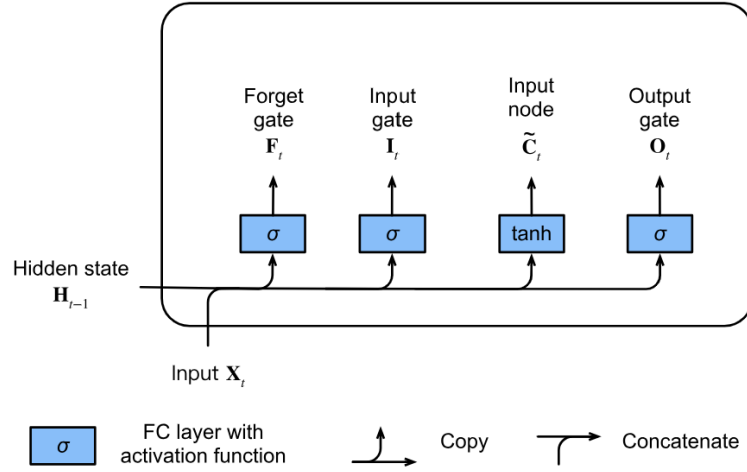


Figure 2.4: Computing the input node in an LSTM model

Memory Cell Internal State

The memory cell state is updated using the forget gate and input gate as:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t$$

where $\mathbf{C}_t$ represents the updated memory cell state and $\odot$ denotes element-wise multiplication. This mechanism helps preserve important long-term information while discarding irrelevant data.

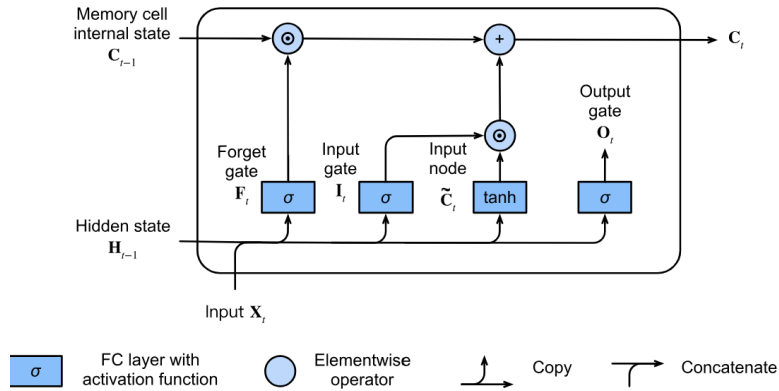


Figure 2.5: Computing the memory cell internal state in an LSTM model

Hidden State

The hidden state output is computed as:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t)$$

where $\mathbf{H}_t$ represents the hidden state at time step t . The output gate controls how much information from the memory cell is passed to the next layer.

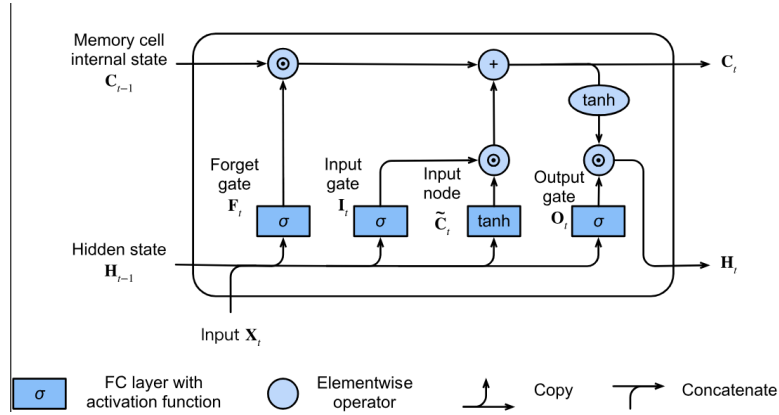


Figure 2.6: Computing the hidden state in an LSTM model

Advantages of LSTM

LSTM effectively solves the vanishing gradient problem and learns long-term sequential dependencies using memory cells and gating mechanisms. It performs efficiently on sequential and time-series data and is widely used in applications such as traffic prediction, speech recognition, machine translation, and forecasting.

Limitations of LSTM

Despite its advantages, LSTM is computationally expensive due to multiple gates and memory operations. Training requires higher computational resources and longer execution time, and performance may decrease for extremely long sequences or insufficient training data.

Source: Adapted from “Dive into Deep Learning, Long Short-Term Memory (LSTM).”

2.3 Federated Learning

Federated Learning (FL) is a decentralized machine learning approach where multiple devices or organizations collaboratively train a shared model without exchanging raw data. Each client performs local training on its own dataset and shares only model parameters with a central server, enabling privacy-preserving collaborative learning across distributed environments.

2.3.1 Introduction

Traditional machine learning systems rely on centralized training, where data from multiple users or organizations is collected and stored on a central server for model development. Although effective, this approach raises significant concerns related to data privacy, security, and regulatory compliance. Federated Learning addresses these challenges by enabling collaborative model training without sharing confidential or sensitive data.

In Trade-Based Money Laundering (TBML) detection systems, organizations such as banks and financial institutions can locally train models using transaction records and suspicious trade patterns while sharing only model parameters with a central server. Introduced by Google in 2016 for Gboard applications, Federated Learning has emerged as an important privacy-preserving approach for distributed machine learning systems.

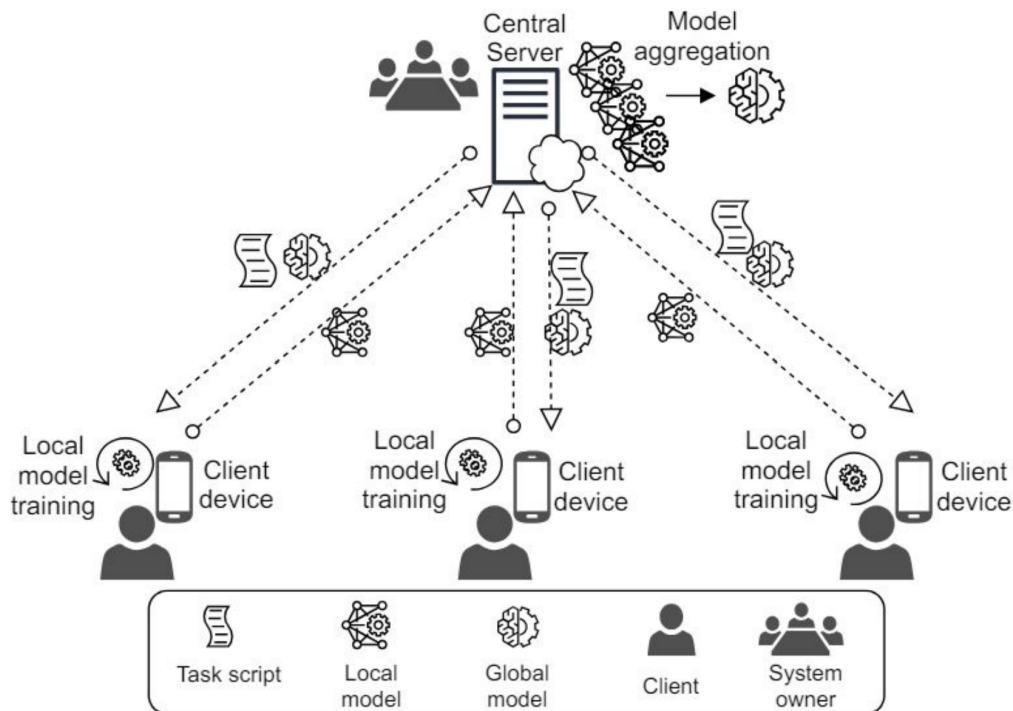


Figure 2.7: General Outlook of Federated Learning

Source: Adapted from “Principles and Components of Federated Learning Architectures” Arxiv.

Federated Learning also reduces communication overhead because only compact model updates are exchanged instead of complete datasets. This makes FL suitable for large-scale distributed environments involving mobile devices, IoT systems, edge devices, and

financial institutions.

2.3.2 Working of Federated Learning

Federated Learning operates through iterative client-server interactions known as federated rounds. During each round, the central server distributes the current global model to selected clients, local training is performed independently on local datasets, and the updated model parameters are aggregated to generate an improved global model.

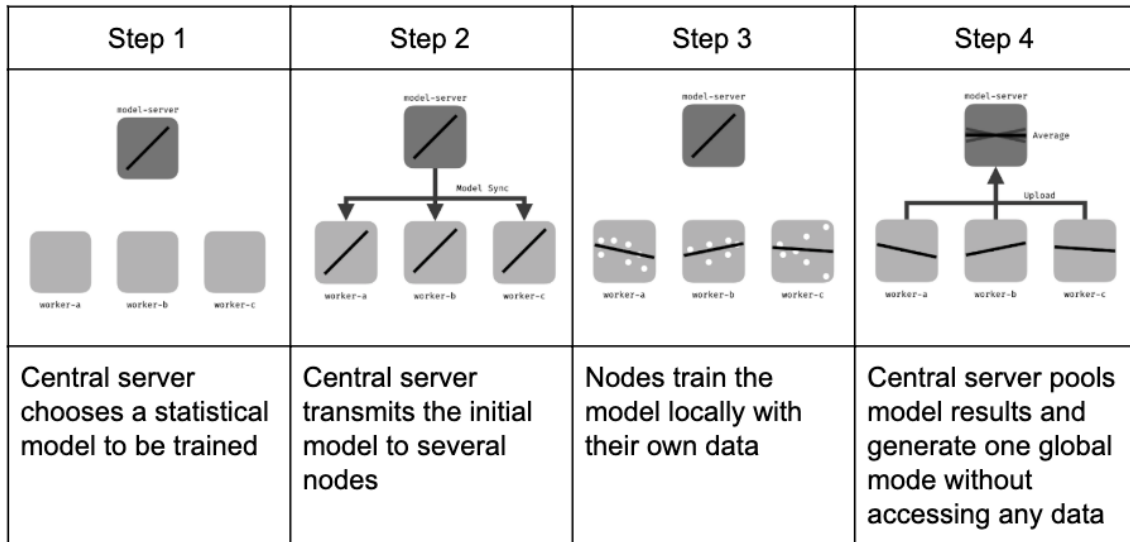


Figure 2.8: Federated Learning General Process in Central Orchestrator Setup

Source: Adapted from “Federated Learning” Wikipedia.

Step 1: Initialization An initial machine learning model is selected and initialized at the central server. Client devices or institutional servers are then prepared to participate in local training.

Step 2: Client Selection During each federated round, only a subset of clients is selected for training. These clients receive the latest global model parameters from the server.

Step 3: Local Training Each selected client trains the received model locally using its private dataset without sharing raw transactional or customer information.

Step 4: Reporting and Aggregation After local training, clients send only model parameters or gradients to the aggregation server. The server combines these updates

using aggregation algorithms such as Federated Averaging (FedAvg).

The Federated Averaging process is represented as:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_k^{(t+1)}$$

where w_{t+1} represents the updated global model, $w_k^{(t+1)}$ denotes the local model of client k , n_k represents the number of training samples at client k , and n denotes the total number of training samples across all clients.

Step 5: Termination The federated training process continues iteratively until the desired accuracy, convergence criteria, or maximum communication rounds are achieved.

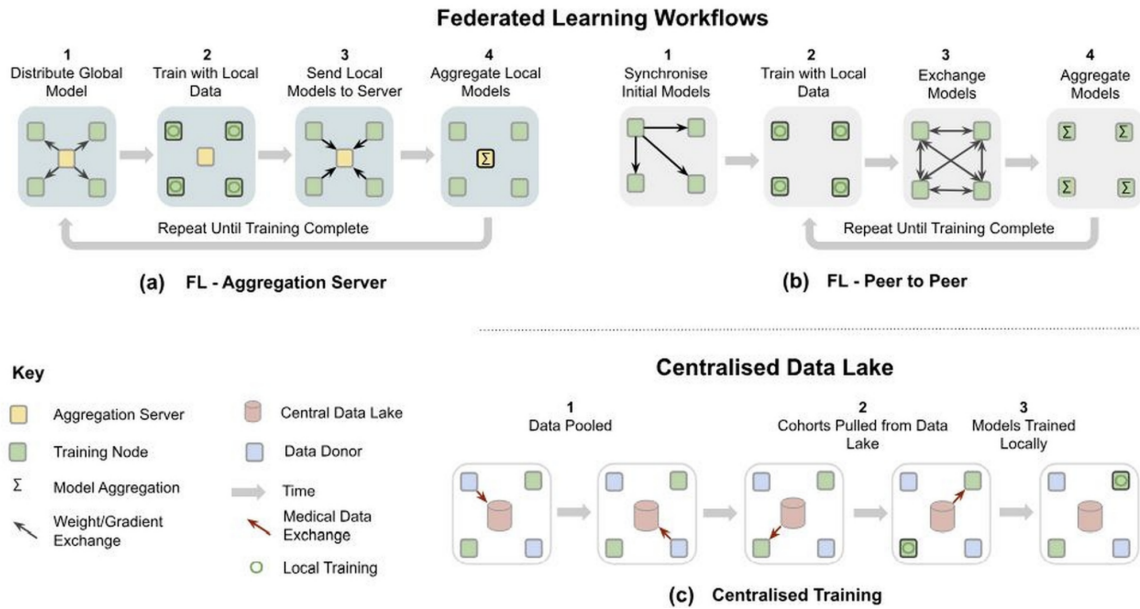


Figure 2.9: Federated Learning Workflows

2.3.3 Advantages of Federated Learning

Federated Learning improves data privacy and security by ensuring that raw data remains within local devices or organizations. It supports regulatory compliance, enables collaborative intelligence without direct data sharing, reduces communication overhead, and improves model generalization using diverse distributed datasets. FL is also scalable and supports real-time learning on edge devices and organizational servers.

2.3.4 Limitations of Federated Learning

Despite its advantages, Federated Learning introduces challenges such as communication delays, high computational requirements, non-i.i.d. data distributions, client failures, synchronization issues, and security threats such as model poisoning and backdoor attacks. Variations in client hardware capabilities and unstable network connections can also affect training efficiency and global model convergence.

2.4 User Interface

Modern financial monitoring platforms increasingly rely on scalable web application technologies to support real-time transaction analysis, fraud monitoring, and interactive visualization of financial activities. The rapid growth of digital banking and online financial services has significantly increased the demand for responsive, secure, and data-driven monitoring systems capable of handling large volumes of transactional information efficiently. Consequently, modern Anti-Money Laundering systems employ component-based web architectures and client-server communication models to provide efficient user interaction and real-time analytical capabilities.

Advanced frontend frameworks and backend service architectures play a critical role in enabling modular application development, dynamic rendering, secure authentication, and seamless integration with fraud detection engines. Such technologies support the visualization of suspicious transactional activities, alert generation, role-based access control, and automated compliance workflows within financial institutions. The integration of scalable user interface technologies with intelligent analytical systems further enhances operational efficiency and supports real-time financial crime investigation processes.

2.4.1 Modern Web Application Frameworks

Modern web application frameworks provide the architectural foundation required for developing scalable, responsive, and interactive software systems. Traditional web applications primarily relied on static page rendering and tightly coupled server-side architectures, which limited dynamic user interaction and real-time data processing capabilities. In contrast, modern frameworks adopt component-based and asynchronous rendering approaches that improve application scalability, maintainability, and user experience within large-scale distributed systems.

Frontend frameworks such as React.js enable the development of reusable and modular

user interface components capable of dynamically updating application states without requiring complete page reloads. Such frameworks employ virtual Document Object Model (DOM) mechanisms and state-driven rendering principles to efficiently manage user interactions and real-time data visualization. This architectural approach significantly improves responsiveness and enables scalable dashboard development for data-intensive applications such as financial monitoring systems.

Server-side rendering and hybrid rendering frameworks such as Next.js further enhance modern web application performance by combining client-side interactivity with optimized server-side content delivery. These frameworks support efficient routing, dynamic data fetching, and improved application scalability while reducing rendering overhead for large-scale systems. Consequently, modern web application frameworks have become an essential foundation for building secure, scalable, and real-time financial monitoring platforms.

2.4.2 Component Lifecycle and Reactivity

Modern web application frameworks employ reactive rendering mechanisms to dynamically synchronize the user interface with application data and user interactions. In component-based architectures such as React and Next.js, the user interface continuously responds to changes in application state, backend responses, and client-side events. This reactive behavior enables efficient rendering of dynamic content while improving scalability, responsiveness, and maintainability within large-scale financial monitoring systems.

The component lifecycle in modern React and Next.js applications generally follows a sequence of rendering, interaction, updating, and cleanup stages. These stages collectively manage how components are initialized, rendered, updated, and removed during application execution.

Server-Side Rendering: Initially, application components are rendered within the server environment where transaction data, analytical summaries, and monitoring information are processed before transmitting the generated content to the client browser. This server-side rendering approach improves initial loading performance and reduces unnecessary client-side computational overhead.

Hydration and Client Initialization: After the rendered content reaches the client browser, React hydrates the interface by attaching event listeners and enabling interactive

functionalities within client-side dashboard components. During this stage, local states, asynchronous API calls, and dynamic monitoring services are initialized to support user interaction and real-time transaction visualization.

Reactive State Updates: As users interact with the dashboard, component states and analytical data continuously update according to backend responses and fraud monitoring events. React utilizes Virtual DOM reconciliation mechanisms to selectively update only the affected dashboard components instead of re-rendering the complete interface.

Re-rendering and Synchronization: The framework dynamically synchronizes updated transaction information, fraud alerts, risk scores, and compliance monitoring analytics with the user interface whenever state transitions occur. This reactive rendering process enables efficient real-time visualization within the proposed Anti-Money Laundering monitoring framework.

Unmounting and Resource Cleanup: When components are removed from the active interface, allocated resources, temporary states, and subscriptions are automatically released to maintain application stability and efficient memory utilization.

Such lifecycle management principles are essential in modern financial monitoring platforms where continuous transactional updates, alert generation, and real-time analytical visualization must be processed efficiently while ensuring scalable and responsive user interaction.

2.4.3 Data Visualization

Understanding the theoretical aspects of data visualization is important for analyzing large volumes of financial transactions and interpreting suspicious transactional behavior within real-time monitoring systems. Visualization of analytical outputs enables financial institutions and compliance analysts to identify fraud patterns, transaction anomalies, risk distributions, and suspicious entity relationships more efficiently when compared with conventional tabular analysis methods. Interactive monitoring dashboards further improve situational awareness by enabling continuous observation of fraud alerts, transaction flows, and compliance activities without interrupting real-time analytical processes.

This work conceptualizes financial monitoring and fraud visualization through a web-based dashboard architecture developed using modern frontend technologies and analytical visualization frameworks.

Frontend Framework: Next.js is employed as the frontend framework for developing responsive and modular user interface components capable of supporting scalable financial monitoring applications. The framework supports server-side rendering, dynamic routing, and efficient integration with backend analytical services operating within the proposed TBML detection platform.

Analytical Visualization Components: Recharts is utilized for visualizing transactional analytics, fraud classifications, risk score distributions, and temporal financial trends through interactive line graphs, bar charts, pie charts, and statistical dashboards. These visualization mechanisms improve analytical interpretation and enable efficient monitoring of suspicious financial activities.

Role-Based Dashboard Interfaces: The dashboard architecture incorporates role-based access mechanisms to provide controlled access to fraud investigation workflows, Suspicious Transaction Report generation, real-time monitoring services, and institutional compliance functionalities according to user privileges and operational responsibilities.

Interactive Monitoring Services: Interactive analytical panels support continuous monitoring of suspicious transactions, transaction histories, fraud alerts, and prediction results generated by intelligent detection modules operating within the framework. The visualization environment additionally supports filtering, notification handling, and compliance-oriented monitoring operations for financial investigators and regulatory analysts.

Modern visualization systems additionally support analytical filtering, export functionalities, and alert management services that assist investigators and compliance officers in conducting financial analysis and generating compliance reports for further examination. The detailed implementation and integration of these visualization technologies within the proposed TBML detection framework are discussed further in the system implementation chapter.

2.5 Summary

In this chapter, the theoretical foundations required for understanding the proposed Trade-Based Money Laundering detection framework were presented. The chapter began with an introduction to Trade-Based Money Laundering mechanisms and common

laundering techniques, establishing the financial and transactional context in which illicit fund movement occurs within international trade ecosystems.

The chapter further explored the challenges associated with detecting suspicious trade activities using conventional Anti-Money Laundering approaches. Concepts related to graph-based financial modeling, heterogeneous graph representation, Graph Neural Networks, Graph Attention mechanisms, and federated learning were discussed to provide the analytical and computational foundation required for intelligent and privacy-preserving fraud detection systems.

Finally, the chapter introduced the concepts associated with modern financial monitoring interfaces, reactive web architectures, and real-time data visualization systems used for fraud analytics and compliance monitoring. The theoretical concepts discussed throughout this chapter provide the foundation for understanding the system architecture, implementation methodologies, and analytical workflows presented in the subsequent chapters.

Chapter 3

Software Requirement Specification

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

The development of an intelligent Trade-Based Money Laundering detection framework requires careful identification of the operational, computational, and functional requirements associated with the system. Since the proposed framework integrates graph-based fraud detection, federated learning, real-time monitoring, and compliance management mechanisms, defining the system requirements becomes essential for ensuring scalability, security, and efficient analytical performance. This chapter presents the Software Requirement Specification (SRS) of the proposed TBML detection framework, including the overall system description, product functionalities, user characteristics, functional and non-functional requirements, and the software and hardware resources required for implementing the proposed system.

3.1 Overall Description

The proposed system is designed as a privacy-preserving financial monitoring platform for detecting suspicious Trade-Based Money Laundering activities within large-scale transactional ecosystems. The framework combines heterogeneous graph-based fraud analysis, federated learning mechanisms, role-based monitoring dashboards, and automated Suspicious Transaction Report (STR) generation within a unified analytical environment. The system supports real-time transaction monitoring, fraud classification, and interactive visualization of suspicious financial activities to assist compliance analysts and financial institutions in identifying hidden laundering patterns efficiently.

3.1.1 Product Perspective

The proposed TBML detection framework functions as an intelligent financial monitoring and fraud analysis platform capable of integrating machine learning-based anomaly detection with interactive compliance monitoring systems. The architecture combines heterogeneous graph representation, Graph Neural Network-based analytical models, federated learning mechanisms, and modern web-based visualization technologies within a unified system environment.

The system represents financial entities such as accounts, organizations, invoices, and transactions as interconnected graph structures for relational analysis. Fraud detection modules analyze these relationships to identify suspicious transactional dependencies

and hidden laundering patterns that are difficult to detect using conventional rule-based Anti-Money Laundering systems. The framework additionally integrates role-based dashboards for transaction monitoring, fraud visualization, alert management, and automated compliance reporting within financial institutions.

3.1.2 Product Functions

The primary functionalities supported by the proposed framework are summarized as follows:

- **Transaction Monitoring:** The system continuously monitors financial transactions and trade-related activities to identify suspicious transactional behavior within large-scale financial ecosystems.
- **Graph-Based Financial Analysis:** Financial entities such as accounts, organizations, invoices, and transactions are represented as interconnected graph structures for relational analysis and hidden dependency detection.
- **Fraud Classification and Risk Prediction:** Graph Neural Network-based analytical models classify suspicious transactions and generate fraud risk predictions based on transactional relationships and behavioral patterns.
- **Federated Learning Support:** The framework supports collaborative model training across multiple institutional environments while preserving sensitive financial data privacy.
- **Real-Time Dashboard Visualization:** Interactive monitoring dashboards provide visualization of transaction analytics, fraud alerts, risk distributions, and suspicious activity summaries for compliance analysts.
- **Suspicious Transaction Report (STR) Generation:** The system automatically generates structured Suspicious Transaction Reports for identified fraudulent transactions to support compliance and regulatory workflows.
- **Authentication and Role-Based Access Control:** Secure authentication and controlled access mechanisms are incorporated to manage user privileges and protect sensitive financial information within the monitoring platform.

3.1.3 User Characteristics

The proposed TBML detection framework is designed to support role-based interaction for users involved in transaction monitoring, fraud investigation, and regulatory compliance activities. Since the system handles sensitive financial information and suspicious transactional records, controlled access mechanisms are incorporated to ensure secure and authorized usage of the platform.

The major user categories supported by the system are summarized as follows:

- **Bank Compliance Team:** The bank compliance team is responsible for monitoring institution-level transactions and analyzing suspicious transactional activities identified by the fraud detection system. Authorized users can review transaction details, examine analytical dashboards, verify fraud predictions, and classify suspicious activities for further investigation. In addition, the compliance team can escalate suspicious transactions as fraudulent, mark transactions for clearance, and generate Suspicious Transaction Reports (STRs) for confirmed fraud cases.
- **Regulatory Administrator:** The regulatory administrator functions as a higher-level monitoring authority with access to suspicious transaction reports generated by participating financial institutions. The administrator can review institution-level fraud cases, analyze consolidated STR records, monitor compliance activities across multiple banks, and oversee large-scale suspicious financial activities within the monitoring ecosystem.

3.1.4 Constraints

The development and deployment of the proposed TBML detection framework are subject to multiple operational, computational, and regulatory constraints associated with financial monitoring systems. These limitations influence the scalability, analytical performance, and real-time operational capabilities of the framework.

The major constraints associated with the proposed system are summarized as follows:

- **Data Privacy and Regulatory Constraints:** Financial institutions operate under strict privacy regulations and compliance policies that restrict direct sharing of sensitive transactional information across organizations.

- **Large-Scale Transactional Data Processing:** The system must process large volumes of interconnected financial transactions and graph relationships, which may increase computational complexity and memory utilization during real-time analysis.
- **Availability of Real-World TBML Datasets:** Access to publicly available Trade-Based Money Laundering datasets is limited due to confidentiality and regulatory restrictions, making large-scale real-world experimentation challenging.
- **Dynamic Fraud Patterns:** Money laundering strategies continuously evolve over time, requiring fraud detection models to adapt to changing transactional behaviors and emerging laundering techniques.
- **Infrastructure and Computational Limitations:** Training graph-based machine learning models and processing heterogeneous financial networks require significant computational resources, especially for large-scale transactional ecosystems.

3.2 Specific Requirements

The specific requirements of the proposed TBML detection framework define the operational functionalities, performance expectations, software dependencies, and computational resources required for efficient execution of the system. These requirements ensure that the framework can support real-time fraud analysis, scalable transaction monitoring, secure user interaction, and intelligent compliance management within financial institutions.

3.2.1 Functional Requirements

The functional requirements define the major operations and services that must be supported by the proposed TBML detection framework. The primary functional requirements of the system are summarized as follows:

- The system shall authenticate authorized users before providing access to monitoring dashboards and analytical services.
- The system shall support role-based access control for bank compliance teams and regulatory administrators.

- The system shall process transactional datasets and construct heterogeneous graph structures representing financial entities and transactional relationships.
- The system shall classify suspicious financial transactions using graph-based analytical models and generate fraud risk predictions.
- The system shall support real-time visualization of transaction analytics, fraud alerts, and suspicious activity summaries through interactive dashboards.
- The system shall allow bank compliance teams to review suspicious transactions and classify them for fraud escalation or clearance.
- The system shall automatically generate Suspicious Transaction Reports (STRs) for identified fraudulent transactions.
- The system shall enable regulatory administrators to monitor fraud reports and suspicious activities generated across participating financial institutions.
- The system shall support analytical filtering and transaction search functionalities for efficient fraud investigation.
- The system shall maintain secure storage and controlled access to sensitive transactional and compliance-related information.

3.2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes, operational constraints, and performance expectations associated with the proposed TBML detection framework. These requirements ensure that the system remains scalable, secure, reliable, and responsive while processing large-scale financial transactions and analytical workloads.

The primary non-functional requirements associated with the proposed system are summarized as follows:

- **Scalability:** The framework should support scalable processing of large transactional datasets and heterogeneous graph structures without significant degradation in analytical performance.

- **Real-Time Processing:** The system should support low-latency transaction analysis and fraud prediction to enable continuous real-time monitoring of suspicious financial activities.
- **Security and Privacy:** The framework should ensure secure access control, protected communication, and privacy-preserving handling of sensitive financial information and compliance-related data.
- **Reliability:** The system should maintain stable analytical performance and continuous monitoring capabilities during prolonged operational workloads and concurrent user access.
- **Availability:** The monitoring platform should provide consistent access to dashboards, fraud analytics, and reporting services for authorized institutional users.
- **Maintainability:** The system architecture should support modular development, efficient debugging, and simplified integration of additional analytical modules and monitoring functionalities.
- **Usability:** The dashboard interfaces and visualization systems should provide intuitive interaction mechanisms for compliance analysts and regulatory administrators during fraud investigation activities.

3.2.3 Software Requirements

The implementation of the proposed TBML detection framework requires multiple software technologies, machine learning libraries, backend services, database systems, and visualization frameworks to support graph-based fraud detection, federated learning, real-time transaction monitoring, and compliance management functionalities. The proposed framework integrates both analytical and monitoring components that require coordinated interaction between frontend dashboards, backend processing services, database systems, and machine learning environments. These technologies collectively support scalable fraud analysis, secure transaction management, and efficient visualization of suspicious financial activities. The software requirements associated with the proposed system are summarized in Table ??.

Table 3.1: Software Requirements of the Proposed TBML Detection Framework

Category	Technologies / Tools	Purpose in the System
Operating System	Windows 11	Provides the development and deployment environment for backend services, databases, and analytical modules.
Programming Languages	Python, JavaScript	Used for backend development, machine learning workflows, API services, and frontend application development.
Frontend Frameworks	Next.js, React.js, Tailwind CSS	Supports responsive user interface development, dashboard visualization, and real-time monitoring functionalities.
Backend Framework	FastAPI	Facilitates asynchronous API communication between frontend dashboards and backend fraud detection services.
Database Systems	PostgreSQL, Memgraph	Stores transactional records, graph relationships, compliance information, and analytical data required for fraud analysis.
Machine Learning Frameworks	PyTorch, PyTorch Geometric	Supports deep learning operations, Graph Neural Network implementation, and graph-driven fraud analysis.
Federated Learning Framework	Flower	Enables distributed federated learning and privacy-preserving model aggregation across institutional environments.
Streaming Platform	Apache Kafka	Supports real-time transaction streaming and event-driven processing within the monitoring pipeline.
Containerization Platform	Docker	Provides containerized deployment and scalable execution environments for system integration.
LLM Integration Service	Groq LLM API	Supports automated generation of Suspicious Transaction Reports and compliance-oriented analytical summaries.

3.2.4 Hardware Requirements

The implementation of the proposed TBML detection framework requires the following hardware components to support graph processing, machine learning model execution, federated learning coordination, real-time transaction monitoring, and analytical dashboard visualization. The hardware requirements associated with the proposed system are summarized in Table ??.

Table 3.2: Hardware Requirements of the Proposed TBML Detection Framework

Hardware Component	Minimum Requirement	Purpose in the System
Processor	Intel Core i7 / AMD Ryzen 7	Supports efficient execution of graph analytical models, backend services, and machine learning computations.
RAM	16 GB	Supports graph datasets, real-time analytical workloads, and concurrent monitoring operations.
GPU	NVIDIA RTX 3050	Accelerates Graph Neural Network training and deep learning computations.
Storage	512 GB SSD	Stores transactional datasets, graph databases, analytical models, and generated compliance reports.

3.3 Summary

In this chapter, the Software Requirement Specification of the proposed Trade-Based Money Laundering detection framework was presented. The operational functionalities, user roles, functional and non-functional requirements associated with the proposed fraud detection platform were discussed to establish the monitoring and analytical capabilities of the system.

The chapter also identified the software technologies, database systems, machine learning frameworks, and hardware resources required for implementing the proposed framework. These specifications collectively establish the operational and computational foundation required for developing the proposed TBML detection system, which is further discussed in the subsequent system design and implementation chapter.

Chapter 4

High Level Design

CHAPTER 4

HIGH LEVEL DESIGN

The design of the proposed Trade-Based Money Laundering detection framework focuses on developing a scalable, privacy-preserving, and intelligent financial monitoring system capable of analyzing complex transactional relationships within large-scale trade ecosystems. Since modern TBML activities involve interconnected financial entities, cross-border transactions, and evolving laundering strategies, the proposed framework integrates graph-based analytical models, federated learning mechanisms, and real-time monitoring services to support efficient fraud detection and compliance management. This chapter presents the major design considerations, architectural strategies, overall system architecture, and data flow mechanisms associated with the proposed fraud detection framework.

4.1 Design Considerations

The design of the proposed TBML detection framework is influenced by multiple operational, analytical, and security-related factors associated with large-scale financial monitoring systems. Since Trade-Based Money Laundering activities involve interconnected financial entities, cross-border transactions, and hidden transactional dependencies, the system architecture must support scalable analytical processing, secure institutional collaboration, and efficient graph-based fraud analysis. The major design considerations associated with the proposed framework are discussed in the following subsections.

4.1.1 General Considerations

The proposed framework is designed to support intelligent monitoring and analysis of large-scale financial transactions within distributed institutional environments. Since suspicious laundering activities often involve interconnected accounts, organizations, invoices, and transactional relationships, the system utilizes graph-based representation mechanisms to model hidden financial dependencies more effectively than conventional relational approaches. The framework additionally supports real-time transaction processing and scalable analytical operations to manage continuously growing financial datasets.

Security and privacy preservation also represent important considerations within the proposed system architecture. Financial institutions operate under strict regulatory policies that restrict unauthorized access and direct sharing of sensitive transactional infor-

mation. Consequently, the framework incorporates secure authentication mechanisms, role-based access control, and federated learning strategies to support protected institutional collaboration and privacy-preserving fraud analysis.

The system is additionally designed using a modular architecture to simplify integration, maintenance, and future scalability of analytical services. Independent modules associated with transaction monitoring, graph construction, fraud detection, compliance reporting, and dashboard visualization operate collaboratively within the overall monitoring framework while maintaining architectural flexibility for future enhancements.

4.1.2 Development Methods

The proposed TBML detection framework is developed using a modular and iterative development methodology to support scalable system integration and continuous refinement of analytical functionalities. Individual system components including transaction monitoring services, graph construction pipelines, fraud detection models, federated learning environments, and dashboard interfaces are developed and tested independently before integration within the complete monitoring framework.

The development process additionally incorporates incremental model evaluation and experimental refinement to improve fraud detection performance and analytical reliability. Graph-based machine learning models are trained using processed transactional datasets, while frontend and backend services are integrated through API-based communication mechanisms to support real-time interaction between monitoring modules and analytical services.

Version control and collaborative development practices are utilized throughout the implementation process to manage backend services, machine learning workflows, frontend interfaces, and analytical configurations efficiently. This modular development methodology simplifies debugging, testing, maintenance, and future enhancement of the proposed fraud detection framework.

4.2 Architectural Strategies

The architectural strategies adopted in the proposed TBML detection framework focus on supporting scalable financial transaction analysis, secure institutional collaboration, and efficient interaction between analytical services and monitoring interfaces. The framework integrates graph-based fraud detection models, backend analytical services,

real-time dashboards, and distributed learning mechanisms within a unified monitoring environment. The major architectural strategies associated with the proposed framework are discussed in the following subsections.

4.2.1 Programming Language

The proposed TBML detection framework primarily utilizes Python for backend services, graph processing, machine learning workflows, and analytical computations. Python was selected due to its extensive support for deep learning libraries, graph analytical frameworks, and federated learning environments. Libraries such as PyTorch, PyTorch Geometric, and Flower are utilized for implementing graph neural network models and distributed learning mechanisms within the fraud detection framework.

The frontend monitoring dashboard is developed using Next.js and React.js to support responsive user interface development and real-time analytical visualization. JavaScript and TypeScript are utilized for implementing dashboard components, transaction monitoring interfaces, and compliance management services. FastAPI is additionally integrated to support API-based communication between frontend dashboards, backend services, and machine learning modules.

4.2.2 System Integration Architecture

The proposed framework follows a modular client-server and service-oriented integration architecture to support coordinated interaction between analytical modules, backend services, and frontend monitoring interfaces. The fraud detection models, transaction monitoring services, authentication modules, and compliance reporting components operate as independent services while maintaining synchronized communication through backend APIs and shared database environments.

The machine learning modules responsible for graph processing and fraud prediction operate independently from the frontend monitoring dashboard. Processed fraud predictions, transaction classifications, and generated Suspicious Transaction Reports are synchronized through backend services to support real-time visualization and compliance management within the monitoring environment. This modular integration strategy simplifies scalability, maintenance, and future extension of the proposed framework.

4.2.3 Data Storage Management

The proposed TBML detection framework utilizes both relational and graph-based database systems to support efficient financial data management and analytical processing. PostgreSQL is utilized for storing structured transactional records, authentication information, compliance-related data, and generated Suspicious Transaction Reports. The relational database environment supports secure storage and efficient retrieval of operational monitoring information within the framework.

Memgraph is utilized as the graph database environment for storing interconnected financial entities and transactional relationships. Accounts, organizations, invoices, and transactions are represented as graph nodes and edges to support relational fraud analysis and graph neural network processing. The combined relational and graph-based storage strategy improves scalability, analytical flexibility, and efficient synchronization between backend analytical services and monitoring dashboards.

4.3 Overall System Architecture

The overall architecture of the proposed TBML detection framework is designed to support intelligent financial transaction monitoring, graph-based fraud analysis, federated learning, and automated compliance reporting within distributed institutional environments. The framework integrates transaction processing services, heterogeneous graph construction mechanisms, machine learning-based fraud detection modules, and real-time monitoring interfaces to identify suspicious transactional relationships associated with Trade-Based Money Laundering activities.

The architecture begins with multiple input sources including banking systems, trade systems, and external financial data providers. Raw transactional records, trade-related documents, KYC information, and institutional monitoring logs are initially processed within the transaction processing module. This stage performs data ingestion, validation, cleansing, normalization, and structured event generation to prepare the data for analytical processing.

The processed data is subsequently transformed into heterogeneous graph structures within the graph construction module. Financial entities such as accounts, organizations, invoices, and transactions are represented as interconnected nodes and edges to capture hidden financial dependencies and suspicious transactional patterns. The generated graph structures consist of node features, edge relationships, and connectivity

information required for graph-based machine learning operations.

The fraud detection layer incorporates a Hybrid Graph Neural Network branch for relational analysis and a temporal learning branch for capturing sequential transaction behavior. These analytical modules process graph structures and institutional transaction sequences to learn complex laundering patterns associated with suspicious activities. The outputs generated from these models are combined to improve fraud representation learning and transaction risk prediction accuracy.

To preserve institutional privacy and regulatory compliance, the framework integrates a federated learning environment in which participating institutions perform localized model training without directly sharing sensitive financial information. Local model updates generated within institutional environments are transmitted to the global aggregation module, where the updates are combined to construct an improved global fraud detection model while maintaining privacy-preserving collaborative learning.

The updated global model is utilized within the inference and risk scoring module to classify suspicious transactions and generate fraud risk scores. Transactions identified as potentially suspicious are forwarded to the compliance management environment for alert generation, Suspicious Transaction Report creation, and regulatory integration support. The overall architecture of the proposed TBML detection framework is illustrated in Figure ??.

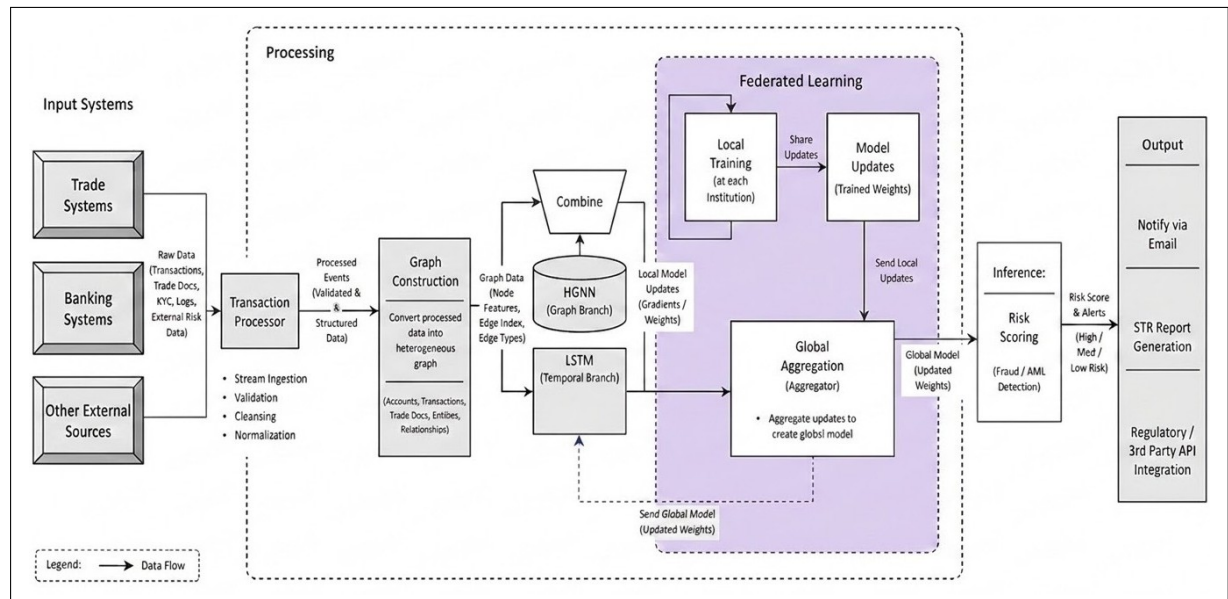


Figure 4.1: Overall Architecture of the Proposed TBML Detection Framework

4.4 Data Flow Diagrams

A Data Flow Diagram (DFD) is a graphical representation used to illustrate the movement of data within a system and the interactions between processes, external entities, and data repositories. DFDs provide a structured visualization of how information enters, is processed, stored, and transferred throughout a system architecture. In the proposed Federated Heterogeneous Graph Neural Network (HGNN)-Based Trade-Based Money Laundering (TBML) Detection System, DFDs are utilized to model the overall workflow of transaction acquisition, graph construction, federated learning, fraud analysis, and alert generation. The hierarchical decomposition of the system into Level 0, Level 1, and Level 2 diagrams enables a detailed understanding of the operational flow, subsystem interactions, and data processing mechanisms involved in real-time AML analytics. The diagrams presented in this section are designed using the Yourdon and De Marco DFD methodology to ensure standardized representation and structured system decomposition.

4.4.1 Data Flow Diagram Level 0

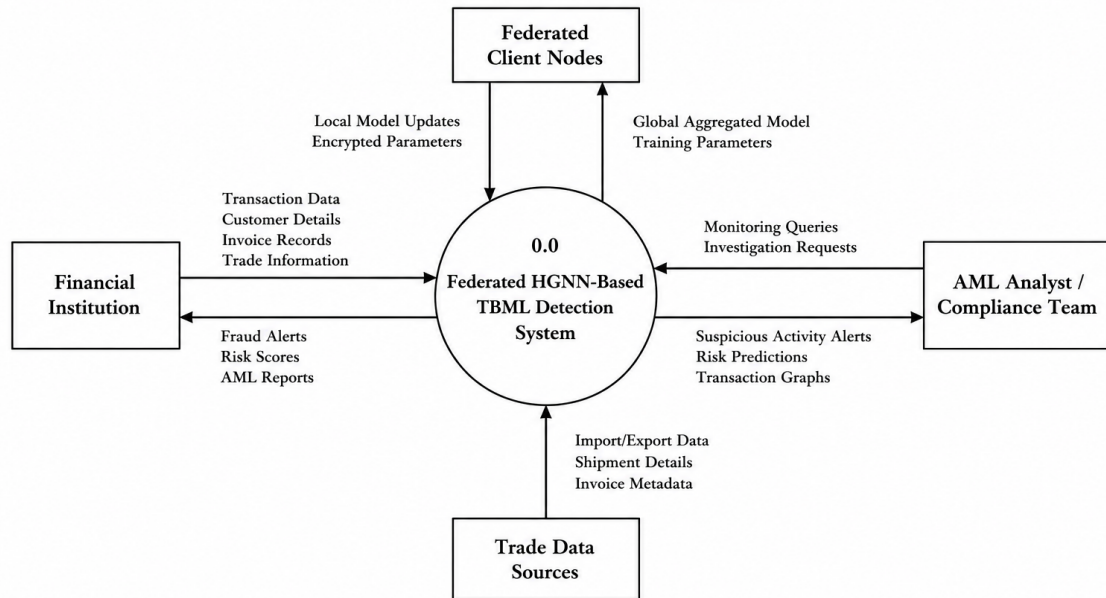


Figure 4.2: Level 0 Data Flow Diagram

Figure 4.2 illustrates the Level 0 Data Flow Diagram of the proposed Federated HGNN-Based TBML Detection System. The diagram provides a high-level contextual representation of the complete system as a single process interacting with external entities including Financial Institutions, Federated Client Nodes, AML Analysts, and Trade Data Sources. Financial institutions provide transaction records, customer information, invoice metadata, and trade-related details to the system for analysis. Trade data sources contribute import-export records and shipment information required for transaction validation and graph construction. Federated client nodes exchange encrypted local model updates and receive aggregated global model parameters during distributed training. The system processes these inputs to generate fraud alerts, risk predictions, AML reports, and suspicious activity notifications for compliance teams and financial analysts. The Level 0 DFD establishes the overall system boundary and illustrates the primary flow of information involved in distributed TBML detection and real-time financial fraud analytics.

4.4.2 Data Flow Diagram Level 1

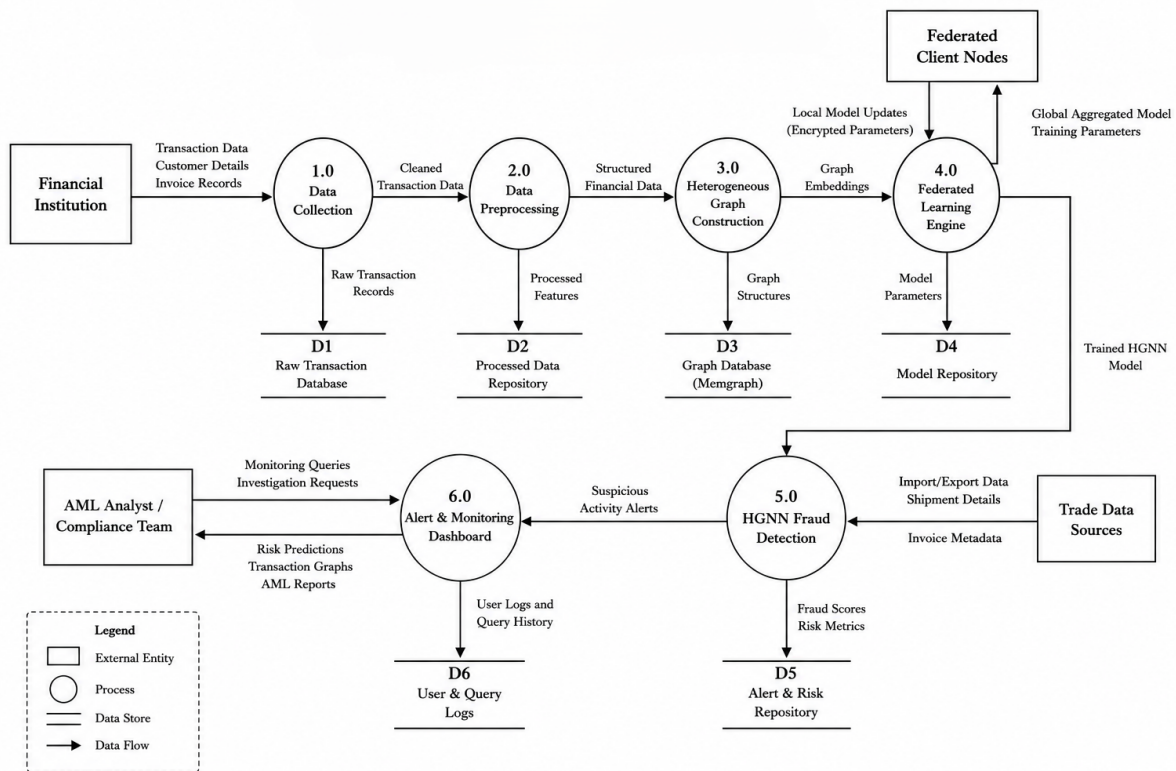


Figure 4.3: Level 1 Data Flow Diagram

Figure 4.3 presents the Level 1 Data Flow Diagram of the proposed TBML detection framework by decomposing the primary system into multiple functional processes. The architecture consists of modules including Data Collection, Data Preprocessing, Heterogeneous Graph Construction, Federated Learning Engine, HGNN Fraud Detection, and Alert and Monitoring Dashboard. The Data Collection module acquires raw transactional records and customer information from financial institutions, which are subsequently cleaned and transformed within the preprocessing stage to generate structured financial features. These processed features are utilized by the graph construction engine to build heterogeneous graph representations stored within the graph database. The federated learning engine performs distributed collaborative model training using encrypted local updates received from federated client nodes while maintaining institutional data privacy. The trained HGNN model is then employed by the fraud detection engine to identify suspicious transactional patterns, calculate fraud scores, and generate anomaly alerts. Finally, the monitoring dashboard enables AML analysts and compliance teams to perform investigations, visualize transaction graphs, and access risk assessment reports for real-time anti-money laundering operations.

4.4.3 Data Flow Diagram Level 2

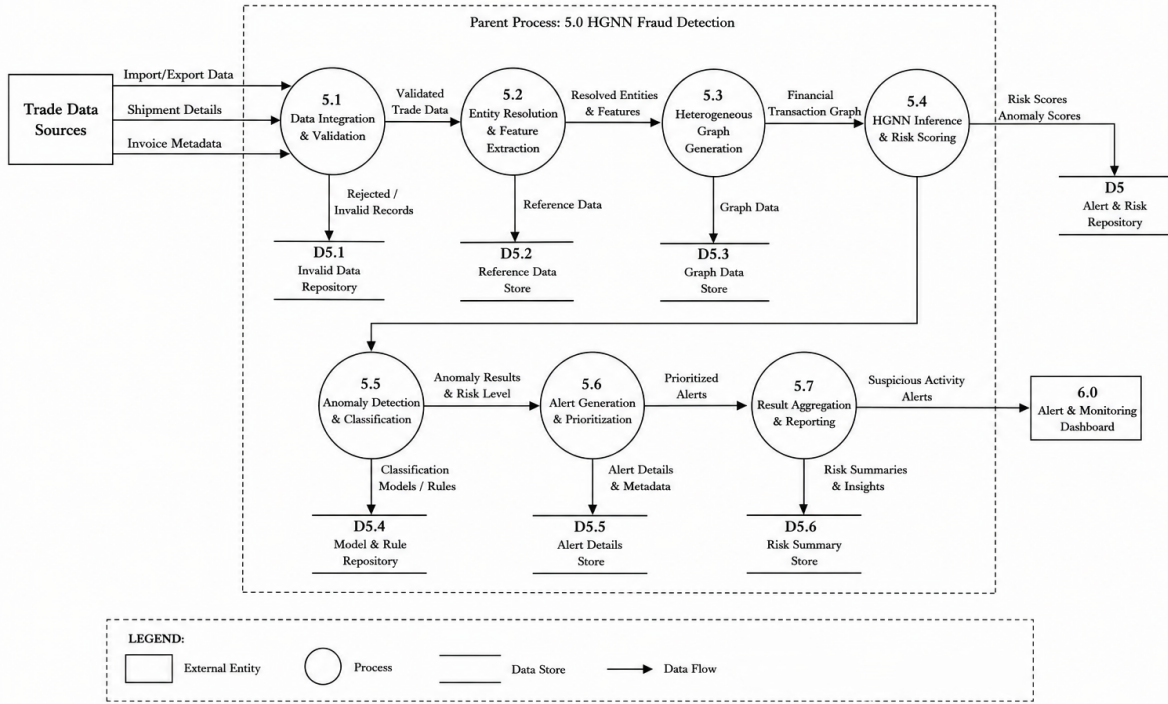


Figure 4.4: Level 2 Data Flow Diagram

Figure 4.4 illustrates the Level 2 Data Flow Diagram representing the internal decomposition of the HGNN Fraud Detection module within the proposed system. The process begins with Data Integration and Validation, where trade invoice records, shipment details, and transactional metadata are verified and filtered for inconsistencies. The validated information is then processed through Entity Resolution and Feature Extraction to identify relational attributes and generate contextual financial features required for graph-based learning. Subsequently, the Heterogeneous Graph Generation process constructs interconnected financial transaction graphs representing entities, accounts, invoices, and trade relationships. The HGNN Inference and Risk Scoring module applies graph neural network-based learning to calculate anomaly scores and identify suspicious transactional behavior. The resulting outputs are further processed through Anomaly Detection and Classification, followed by Alert Generation and Prioritization to rank suspicious activities according to calculated risk severity. Finally, the Result Aggregation and Reporting module generates consolidated fraud summaries, suspicious activity alerts, and analytical reports that are forwarded to the Alert and Monitoring Dashboard for in-

vestigation and compliance analysis. The Level 2 DFD provides a detailed representation of the internal analytical workflow employed by the proposed federated graph intelligence framework for real-time TBML detection.

4.5 Summary

This chapter presented the high-level design of the proposed Federated Heterogeneous Graph Neural Network (HGNN)-Based Trade-Based Money Laundering (TBML) Detection Framework. The design considerations focused on scalability, privacy preservation, modularity, real-time transaction monitoring, and secure institutional collaboration within distributed financial environments. The chapter also discussed the development methodology, architectural strategies, programming technologies, and system integration mechanisms utilized for implementing graph-based fraud analysis, federated learning, and compliance monitoring services.

The overall system architecture demonstrated the integration of transaction processing modules, heterogeneous graph construction, hybrid graph neural network models, federated collaborative learning, and compliance reporting environments for intelligent TBML detection. Additionally, the chapter presented the Level 0, Level 1, and Level 2 Data Flow Diagrams to illustrate the movement of financial data, internal analytical workflows, graph-based fraud detection processes, and alert generation mechanisms within the proposed framework. These architectural components collectively establish a scalable, privacy-aware, and intelligent monitoring system for real-time anti-money laundering analysis.

Chapter 5

Detailed Design of the Proposed TBML Framework

CHAPTER 5

DETAILED DESIGN OF THE PROPOSED TBML FRAMEWORK

The detailed design of the proposed TBML detection framework focuses on the internal organization, analytical workflow, and module-level interaction associated with graph-based fraud detection and real-time financial monitoring. The framework integrates transaction processing services, heterogeneous graph construction, hybrid HGNN–LSTM analytical models, federated learning mechanisms, and compliance reporting modules within a unified monitoring environment. This chapter presents the structure chart, detailed module design, database schema organization, and monitoring workflow associated with the proposed TBML detection framework.

5.1 Structure Chart

The structure chart of the proposed TBML detection framework illustrates the hierarchical organization of the major functional modules operating within the system. The framework is divided into frontend, backend, fraud detection, federated learning, database, and compliance layers to support scalable financial monitoring and intelligent fraud analysis within distributed Anti-Money Laundering environments.

The frontend layer provides dashboard interfaces, transaction monitoring services, risk visualization, and STR management functionalities for institutional users. Backend services coordinate API communication, transaction processing, authentication workflows, and access control operations between frontend interfaces and analytical modules. The fraud detection layer incorporates Hybrid HGNN–LSTM components responsible for graph-based analysis, temporal transaction modeling, feature fusion, and risk classification.

The federated learning layer enables privacy-preserving collaborative model training and global model synchronization between participating institutions. Database services manage transactional storage, graph relationships, audit logs, and compliance-related records, while the compliance layer supports STR generation, regulatory reporting, email notifications, and monitoring operations. The overall structural organization of the proposed TBML detection framework is illustrated in Figure ??.

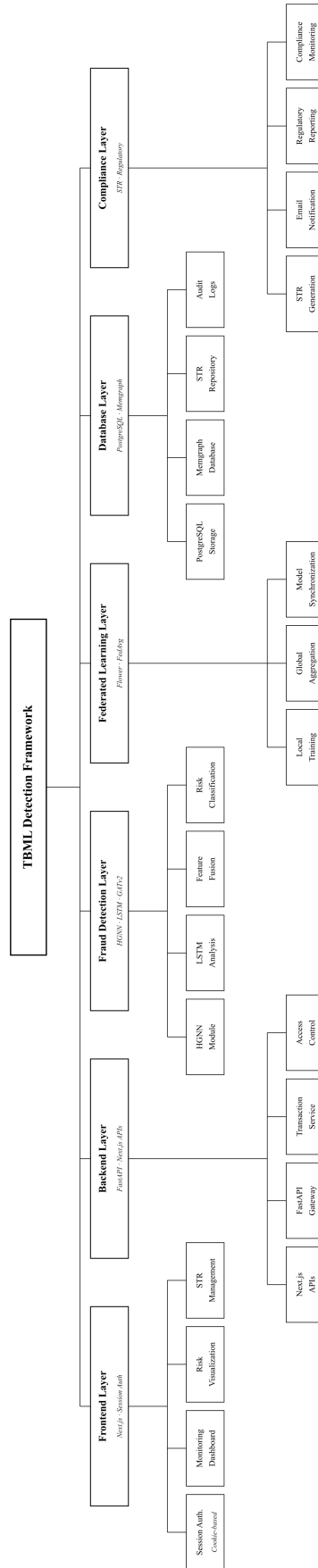


Figure 5.1: Structure Chart of the Proposed TBML Detection Framework

The structure chart illustrates the modular organization and interaction between analytical, monitoring, storage, and compliance-oriented services within the proposed framework. This modular design approach improves scalability, maintainability, service integration, and efficient coordination between frontend interfaces, backend analytical services, and distributed fraud detection environments.

5.2 Functional Description of the Modules

The functional description of the modules explains the internal processing workflow and interaction between the major analytical, monitoring, and compliance-oriented components operating within the proposed TBML detection framework. The framework integrates transaction streaming services, graph construction mechanisms, hybrid HGNN–LSTM analytical models, institutional verification environments, and regulatory reporting systems to support intelligent Trade-Based Money Laundering detection and real-time compliance monitoring.

5.2.1 Module Interaction Workflow

The module interaction workflow illustrates the coordinated operational flow between transaction ingestion services, streaming environments, graph-based analytical modules, fraud classification mechanisms, institutional verification procedures, and regulatory monitoring systems functioning within the proposed framework. Financial transactions, KYC records, and trade documents are initially processed within the transaction processing layer and published into Kafka streaming topics for asynchronous event-driven communication between backend analytical services.

Kafka consumer services subsequently consume the transaction streams and forward the processed data to the graph construction and feature extraction module where heterogeneous financial transaction graphs are generated using accounts, transactions, trade entities, and document relationships stored within the Memgraph database. The extracted graph embeddings and relational features are processed by the Hybrid HGNN–LSTM fraud detection module to perform graph representation learning, temporal transaction analysis, and intelligent fraud prediction.

The generated fraud classifications and monitoring results are stored within the PostgreSQL database and visualized through the dashboard monitoring environment for institutional investigators and banking administrators. Suspicious transactions identified

by the analytical system are reviewed by authorized bank administrators before being categorized as clean, suspicious, or confirmed fraudulent transactions.

Transactions verified as fraudulent are forwarded to the STR generation environment where Suspicious Transaction Reports are generated with analytical assistance from the Groq API service. Generated STR reports are subsequently reviewed by regulatory authorities responsible for investigation management, inquiry procedures, and enforcement operations. Notification management services coordinate fraud alerts and institutional communication processes, while the Resend email service delivers notification emails and regulatory enforcement messages to the corresponding institutional authorities. The overall module interaction workflow of the proposed TBML detection framework is illustrated in Figure ??.

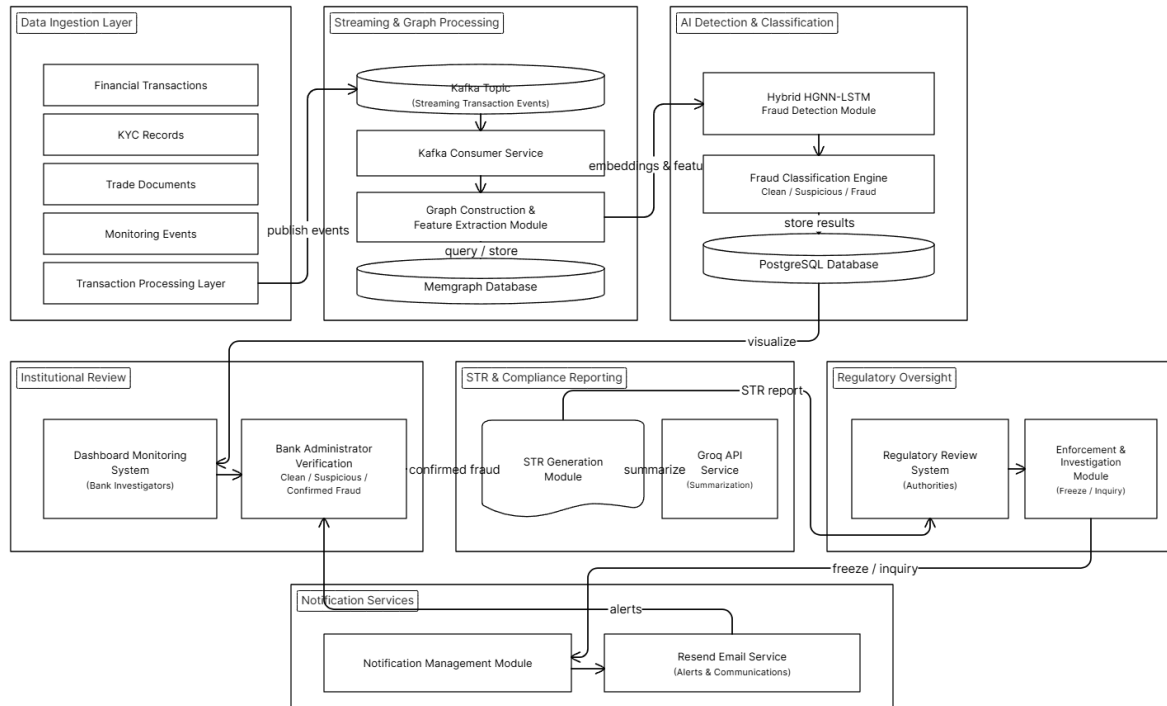


Figure 5.2: Workflow Interaction Diagram for the Proposed TBML Detection Framework

The workflow interaction diagram illustrates the coordinated interaction between streaming services, graph-based analytical modules, monitoring environments, institutional verification systems, and regulatory compliance services operating within the proposed framework. Such modular workflow integration supports scalable transaction processing, intelligent fraud analysis, compliance management, and real-time Anti-Money Laundering monitoring within distributed financial environments.

5.2.2 Transaction Processing Module

The transaction processing Module acts as the initial ingestion environment within the proposed TBML detection framework. This Module collects financial transactions, KYC records, trade documents, and institutional monitoring events originating from banking systems and external financial environments.

The input data processed within this Module includes sender and receiver account identifiers, transaction amounts, commodity types, trade quantities, trade values, invoice details, transaction timestamps, and institutional monitoring attributes associated with financial activities. The collected transactional data is validated, normalized, and converted into structured transaction records to support reliable downstream analytical processing.

The processed transaction events are published into Kafka streaming topics to enable asynchronous communication between backend services and analytical modules. The generated transaction streams serve as the primary input for graph construction, feature extraction, and graph-based fraud detection operations within the proposed framework.

5.2.3 Graph Construction Module

The graph construction module transforms processed transaction streams into heterogeneous financial transaction graphs for downstream graph-based analytical processing. The module receives validated transaction events from Kafka consumer services and models financial interactions between accounts, transactions, and trade documents within a connected graph environment.

The generated graph schema consists of interconnected node entities including (:Account), (:Transaction), and (:Document) connected through relationships such as [:SENDS], [:TO], and [:HAS_DOC] as illustrated in Figure ???. The heterogeneous graph structure preserves transactional connectivity, intermediary account interactions, and trade-document relationships required for identifying hidden financial dependencies associated with Trade-Based Money Laundering activities.

The module interacts with the Memgraph database to store graph structures, entity relationships, and transactional connectivity information required for downstream feature extraction and fraud analysis. The generated graph representations and extracted relational features are subsequently forwarded to the Hybrid HGNN–LSTM fraud detection module for intelligent transaction classification.

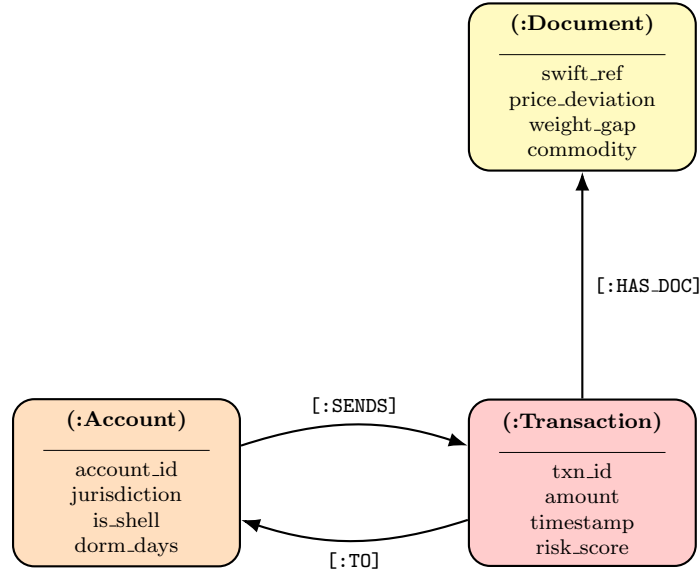


Figure 5.3: Heterogeneous graph schema representing entity attributes and transactional relationships in the proposed TBML detection framework.

5.2.4 Hybrid HGNN–LSTM Fraud Detection Module

The Hybrid HGNN–LSTM fraud detection module performs graph-based representation learning and temporal transaction analysis within the proposed TBML detection framework. The module receives KYC node features, graph edge relationships, SWIFT transaction attributes, sequential transaction tensors, and trade metadata extracted from the heterogeneous financial graph for downstream analytical processing.

The graph branch utilizes a three-layer GATv2 architecture to perform attention-based neighborhood aggregation and multi-hop message passing across interconnected financial entities. The stacked GATv2 layers improve two-hop neighborhood visibility and enable the model to capture hidden transactional relationships and intermediary account interactions associated with Trade-Based Money Laundering activities. The generated graph embeddings are globally pooled and fused with sequential transaction representations before being processed through the LSTM branch for temporal anomaly detection and transaction behavior analysis. In parallel, the trade branch processes trade-specific attributes including price deviation and weight gap scores using multilayer perceptron operations and feature normalization layers.

The generated graph embeddings, temporal representations, and trade feature vectors are combined within the feature fusion layer for downstream fraud prediction and transaction risk analysis. The final dense classification layer categorizes transactions into

predefined classes including CLEAN, WATCHLIST, and FRAUD. The overall architecture of the proposed Hybrid HGNN–LSTM fraud detection module is illustrated in Figure ??.

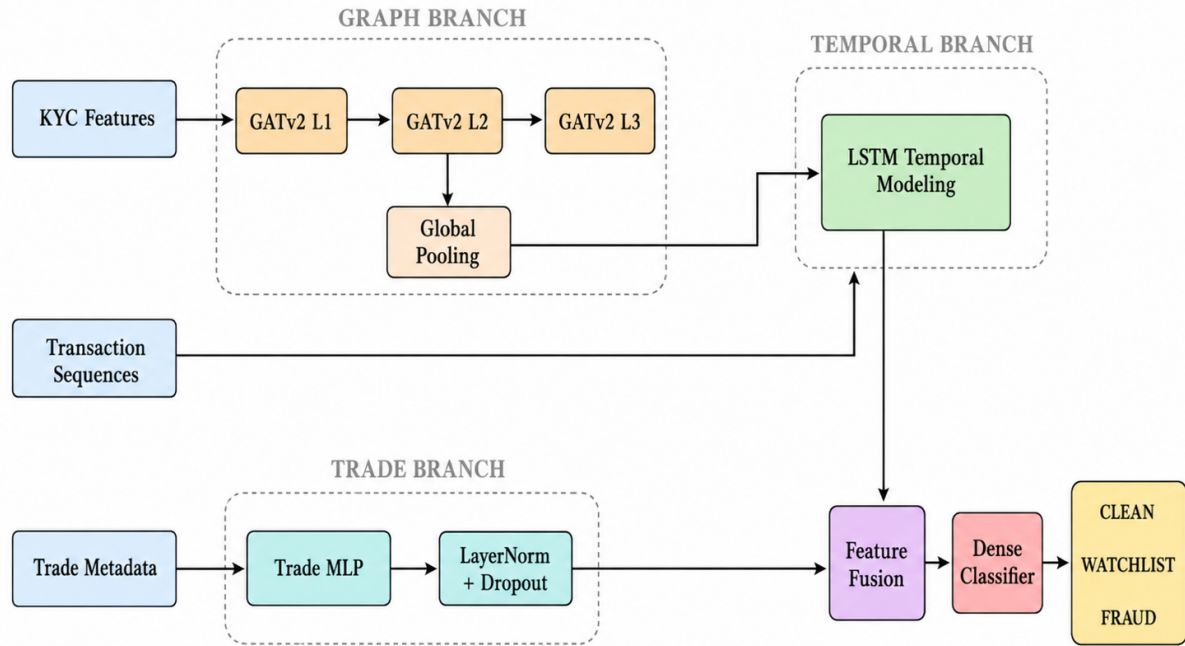


Figure 5.4: Architecture of the Proposed Hybrid HGNN–LSTM Fraud Detection Module

5.2.5 Federated Learning Module

The federated learning module enables privacy-preserving collaborative fraud detection across distributed financial institutions without directly sharing sensitive transactional data. Each participating institution performs local model training using institutional transaction graphs, transaction sequences, and fraud classification labels generated within the analytical environment. Independent federated clients maintain isolated training environments and perform localized optimization of the Hybrid HGNN–LSTM model on institution-specific financial datasets.

The framework utilizes the Federated Averaging (**FedAvg**) strategy to aggregate distributed model parameters across participating institutions. During each federated communication round, locally trained model weights generated from individual institutional clients are transmitted to the central aggregation server where parameter averaging operations are performed to generate a globally optimized fraud detection model. The aggregated global model parameters are subsequently redistributed to participating institutions for synchronized fraud detection and continuous real-time analytical inference.

The overall federated learning architecture of the proposed framework is illustrated in Figure ??.

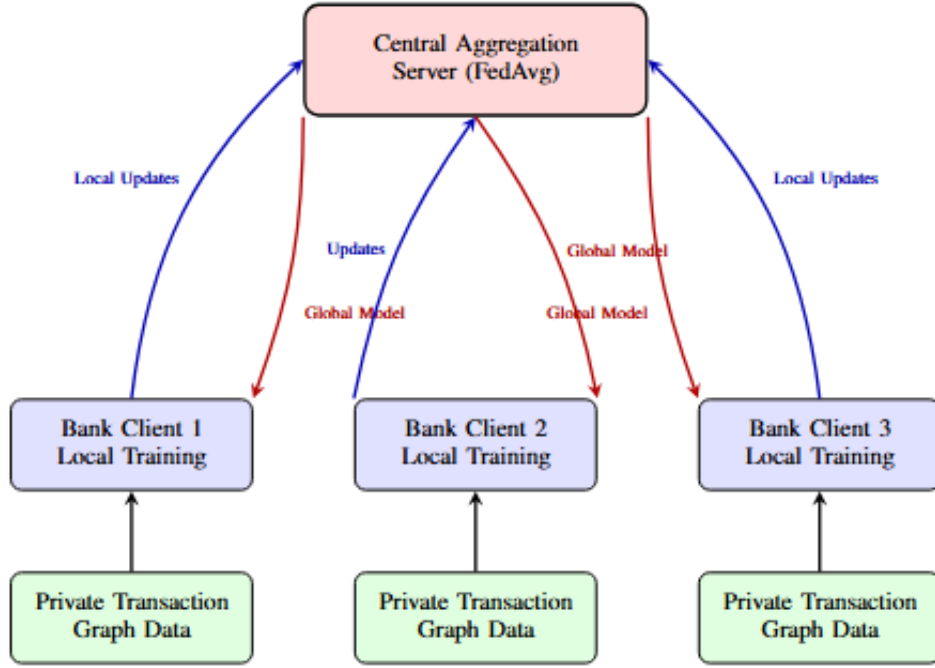


Figure 5.5: Federated learning architecture for privacy-preserving collaborative TBML detection across distributed institutions

5.2.6 STR Generation and Notification Module

The STR generation and notification module is responsible for compliance reporting and fraud alert management within the proposed TBML detection framework. Transactions verified as fraudulent by authorized banking administrators are forwarded to this module for Suspicious Transaction Report (STR) generation and regulatory processing. The module generates STR reports containing transaction details, fraud classifications, trade metadata, institutional information, and investigation summaries associated with suspicious financial activities.

The Groq API service is utilized for analytical summarization and automated assistance during compliance documentation generation. Generated STR reports are subsequently forwarded to regulatory monitoring authorities for investigation and enforcement procedures. The notification management environment coordinates fraud alerts and institutional communication workflows, while the Resend email service delivers notification emails, investigation alerts, and regulatory enforcement messages to banking institutions.

and regulatory authorities.

5.2.7 Dashboard and Regulatory Monitoring Module

The dashboard and regulatory monitoring module provides real-time transaction visualization, fraud monitoring, and institutional investigation functionalities within the proposed TBML detection framework. The monitoring dashboard displays transaction classifications, fraud alerts, risk scores, STR records, and analytical monitoring information generated by the fraud detection environment. Separate dashboard environments are maintained for banking administrators and regulatory authorities to support institution-level monitoring and centralized compliance management operations.

Authorized banking administrators can view suspicious and fraudulent transactions identified by the analytical model and perform institutional verification procedures before categorizing transactions as **CLEAN**, **WATCHLIST**, or **FRAUD**. Suspicious transactions can either be escalated as fraudulent transactions or marked as clean after institutional verification procedures, thereby reducing false positive classifications within the monitoring environment. Transactions verified as fraudulent are subsequently forwarded for Suspicious Transaction Report (STR) generation and regulatory reporting.

Regulatory authorities monitor generated STR reports and institution-level fraud cases through centralized monitoring dashboards. The regulatory environment supports compliance monitoring, document verification requests, inquiry procedures, and account freeze operations associated with suspicious financial activities.

5.3 Summary

This chapter presented the detailed design of the proposed TBML detection framework including the structural organization, module interaction workflow, functional description of analytical modules, federated learning architecture, and database management environment associated with the system. The chapter discussed the interaction between transaction processing services, graph construction mechanisms, hybrid HGNN–LSTM analytical modules, dashboard monitoring environments, and compliance reporting systems operating within the proposed framework.

The detailed design further illustrated how streaming-based transaction processing, graph-based fraud analysis, federated collaborative learning, institutional verification, STR generation, and regulatory monitoring are integrated within a unified Anti-Money

Laundrying environment. The subsequent chapter discusses the implementation details, backend integration and dashboard development associated with the proposed TBML detection framework.

Chapter 6

Implementation

CHAPTER 6

IMPLEMENTATION

This chapter describes the implementation of the FedGAT-TBML framework: real-time transaction streaming, heterogeneous graph construction, a hybrid HGNN-LSTM detector, federated learning, and a dashboard for compliance monitoring. The system is implemented using Apache Kafka, Memgraph, PostgreSQL, PyTorch Geometric, Flower, FastAPI, and Next.js to enable scalable, real-time TBML detection.

6.1 Programming Language and Framework Selection

The implementation primarily uses Python for backend services and model training (PyTorch + PyTorch Geometric — e.g., `GATv2Conv` for graph attention), Flower for federated orchestration (custom `FedAvg` strategy), and Apache Kafka for durable streaming. Frontend and UI services use Next.js, TypeScript and Tailwind; FastAPI provides REST endpoints. Memgraph stores heterogeneous transaction graphs and PostgreSQL holds audit logs and STR records. Services are containerized with Docker for reproducible deployment.

6.2 Platform Selection

The proposed TBML detection framework is developed within a Windows-based environment consisting of interconnected frontend monitoring services and backend analytical infrastructures. The frontend platform manages authentication workflows, dashboard visualization, STR generation, regulatory monitoring, and institutional transaction management, while the backend analytical environment performs transaction streaming, graph generation, fraud inference, and federated model synchronization operations.

The implementation utilizes Docker-based isolated runtime environments for Kafka services, graph databases, backend APIs, and federated learning modules to improve deployment consistency and distributed service orchestration. Memgraph is utilized for graph relationship management and transactional connectivity analysis, whereas PostgreSQL functions as the centralized synchronization database between analytical services and dashboard environments. Visual Studio Code and GitHub are utilized for collaborative development, source-code management, debugging, and version control operations

throughout the implementation lifecycle.

6.3 Code Conventions

The codebase follows concise, practical conventions to improve readability and maintenance. Python modules and functions use **snake_case**; classes and UI components use **PascalCase**. Examples of graph entities and relationships include `(:Account)`, `(:Transaction)`, `(:Document)` and `[:SENDS]`, `[:TO]`, `[:HAS_DOC]`. REST endpoints follow clear, resource-oriented naming (e.g., `/api/transactions/{id}/score`). Configuration is managed via `.env` files, logging uses structured JSON logs, and type hints plus docstrings are used where helpful.

6.3.1 Naming Conventions

Naming conventions within the proposed framework are designed to remain semantically meaningful and aligned with domain-specific financial monitoring operations. Python variables, backend utility functions, Kafka services, and transaction-processing modules follow **snake_case** conventions, whereas analytical classes, federated learning modules, and frontend components utilize **PascalCase** notation. REST API endpoints, transaction services, and graph-processing utilities are named according to their operational functionality to improve implementation readability and maintainability.

Graph entities, transactional relationships, and database schemas additionally follow consistent naming conventions across graph and relational database environments. Examples include graph entities such as `(:Account)`, `(:Transaction)`, and `(:Document)`, along with transactional relationships including `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]` utilized throughout graph construction and fraud analysis operations.

6.3.2 File Organization

The proposed TBML detection framework follows a modular project structure where frontend monitoring services and backend analytical environments are maintained as interconnected distributed applications. The frontend environment contains dashboard interfaces, authentication services, STR management workflows, API handlers, reusable UI components, and regulatory monitoring modules implemented using the Next.js App Router architecture and TypeScript-based service organization.

The backend analytical environment contains Kafka producers and consumers, graph construction services, Hybrid HGNN-LSTM model implementations, federated learning

clients, inference pipelines, and database synchronization utilities responsible for real-time fraud analysis and transaction monitoring operations. Shared configuration files, utility services, logging environments, and centralized database connectors are additionally organized according to functional responsibilities to improve scalability, maintainability, and distributed service coordination across the proposed framework.

6.3.3 Environment Configuration and Logging

Environment variables and runtime configurations are securely managed using centralized `.env` configuration files to prevent direct exposure of API credentials, Kafka broker addresses, database configurations, authentication tokens, and federated learning parameters within the codebase. Backend services including Kafka consumers, Memgraph connectors, PostgreSQL clients, and fraud inference pipelines are initialized during runtime to support centralized service configuration and distributed analytical synchronization.

Logging mechanisms are integrated throughout transaction streaming services, graph-processing environments, federated learning modules, backend APIs, and fraud inference pipelines to support runtime monitoring, debugging, audit traceability, and compliance management operations. Fraud classifications, transaction verdicts, confidence scores, STR generation events, and institutional monitoring activities are additionally logged within PostgreSQL environments for dashboard synchronization and regulatory investigation workflows.

6.4 Difficulties Encountered and Strategies Used to Tackle Them

Implementation challenges included noisy and imbalanced labels, large graph sizes and memory constraints, streaming latency, and privacy-preserving coordination across institutional clients. Mitigations applied in the implementation include class rebalancing and targeted data augmentation for scarce labels, subgraph sampling and mini-batching to control memory, Kafka buffering and asynchronous consumers to reduce end-to-end latency, and Flower with secure aggregation and containerized deployments (Docker) to isolate client environments while enabling federated training.

6.5 System Implementation

The system implementation of the proposed TBML detection framework follows a distributed event-driven architecture integrating Kafka-based transaction streaming, heterogeneous graph generation, Hybrid HGNN–LSTM analytical processing, federated collaborative learning, real-time fraud inference, and dashboard-driven compliance monitoring. Backend analytical services, graph databases, inference pipelines, and frontend monitoring environments communicate through synchronized APIs, Kafka streams, and centralized PostgreSQL audit infrastructures to support scalable real-time Trade-Based Money Laundering detection and institutional monitoring operations.

6.5.1 Methodology of Synthetic Dataset Synthesis

The generation of the synthetic dataset is engineered to emulate the multi-modal complexities inherent in international trade finance. The process initializes by constructing a baseline network of financial entities—Accounts, Documents, and Transactions—governed by power-law distributions to simulate realistic institutional transaction volumes. Subsequent stages facilitate the injection of complex temporal patterns, commodity-specific price anomalies, and multi-currency layering effects, creating a high-fidelity environment for model training.

The underlying implementation utilizes a modular pipeline that leverages the Pandas library for vectorized data manipulation and Memgraph for graph-structured persistence. Each transaction record is anchored to its corresponding trade documentation through a schema-aligned bridge, ensuring that the structural connectivity required for Graph Neural Network (GNN) feature extraction is preserved. By integrating temporal constraints and noise-injection modules, the pipeline creates a realistic representation of financial behaviors, ranging from standard legitimate interactions to camouflaged, high-velocity laundering topologies.

6.5.2 Adversarial Refinement for Dataset Fidelity

To ensure the framework develops generalized representations rather than relying on heuristic shortcuts, the synthetic generation process underwent significant adversarial refinement. The following modifications were implemented to enhance the dataset's empirical realism and mitigate unintended machine learning biases:

1. **Price Multiplier Dynamics:** Replaced static price multipliers with Gaussian

Mixture Models (GMMs) and stochastic blending, ensuring 30% of fraudulent trades mimic normal market pricing to prevent simple threshold detection.

2. **Elimination of Feature Bias:** Removed the *port_log_status* field to prevent "ground truth leakage," compelling the GNN to perform holistic structural analysis rather than relying on explicit categorical tags.
3. **KYC/Dormancy Engineering:** Implemented overlapping dormancy windows and noise injection to create realistic feature distributions, ensuring dormancy periods serve as contributing factors rather than binary identifiers.
4. **Universal Topology Injection:** Applied complex graph topologies (e.g., Cycles, Fan-In, Gather-Scatter) across all transaction classes to eliminate "Guilty by Shape" bias.
5. **Refined Pattern Classification:** Removed label-based tags (LEGIT/SUSPICIOUS) and introduced "No_Pattern" as a valid structural classification, decoupling structural role from risk label.
6. **Temporal Realism:** Anchored the dataset to a one-year production horizon (Dec 2024–Dec 2025) and enforced realistic time-horizons (Clean: 1–4 weeks; Fraud: 3–6 months).
7. **Scale-Free Network Topology:** Introduced a Pareto/Zipf distribution to simulate "Whale" nodes, creating high-throughput backgrounds that camouflage fraudulent structural patterns.
8. **Log-Normal Transaction Amounts:** Modeled transaction values using a Log-Normal distribution to replicate real-world financial distributions, moving away from uniform, synthetic-sounding values.
9. **Multi-Hop Structural Integrity:** Enforced rigorous, rule-based generation for multi-hop topologies to maintain structural consistency across the graph.
10. **FX Settlement/Currency-Jump Awareness:** Incorporated multi-currency flows and FX spread monitoring to sensitize the model to TBML layering tactics.

11. **Stochastic Trade Price Blending:** Blended the distribution of illicit prices to remove detectable tabular shortcuts, compelling the model to utilize context-aware inference.
12. **Data Leakage Mitigation:** Systematically removed auxiliary flags that offered deterministic solutions to non-deterministic laundering challenges.
13. **Endurance-Based Simulation:** Refined the generation logic to ensure that multi-hop cycles represent sustained operational behaviors rather than instantaneous transactional artifacts.

6.5.3 Kafka-Based Transaction Streaming

The proposed TBML detection framework utilizes Apache Kafka to support asynchronous transaction streaming and event-driven communication between distributed analytical services. A Kafka producer continuously generates synthetic SWIFT transaction payloads containing sender identifiers, receiver identifiers, transaction amounts, commodity information, trade quantities, price deviation scores, and weight-gap attributes before publishing them into the `incoming_swift_messages` topic for downstream analytical processing.

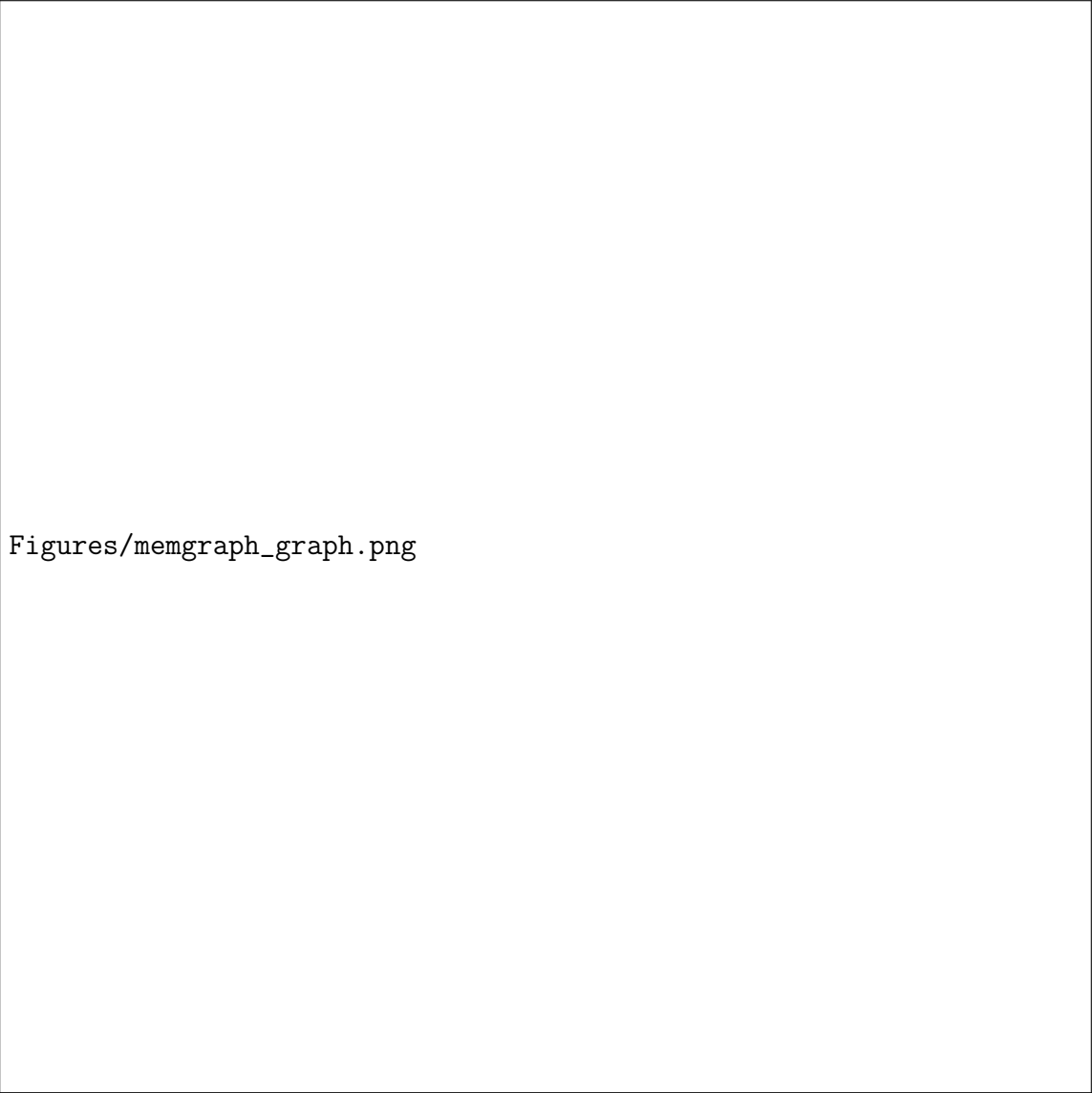
Kafka consumer services continuously subscribe to the transaction stream and process incoming financial events in real time for graph generation and fraud inference operations. The consumer pipeline validates transaction payloads, constructs heterogeneous graph relationships within Memgraph, extracts multi-hop transactional subgraphs, and forwards graph tensors to the Hybrid HGNN–LSTM inference engine for downstream fraud prediction and PostgreSQL audit logging. The streaming architecture enables decoupled communication between frontend services, graph-processing environments, and distributed analytical modules operating within the proposed framework.

6.5.4 Graph Construction and Feature Extraction

The graph construction pipeline transforms streamed financial transactions into heterogeneous graph representations for downstream graph-based analytical processing. Incoming transaction payloads consumed from the `incoming_swift_messages` Kafka topic are processed using Cypher-based graph ingestion operations within Memgraph, where transactional entities and financial relationships are dynamically mapped into intercon-

nected graph structures as illustrated in Figure ??.

The generated graph schema consists of heterogeneous node entities including (:Account), (:Transaction), and (:Document) connected through relationships such as [:SENDS], [:TO], and [:HAS_DOC]. The feature extraction pipeline subsequently performs constrained multi-hop subgraph traversal around the target transaction to generate node feature tensors, graph edge mappings, sequential transaction representations, and trade metadata tensors required for Hybrid HGNN–LSTM-based fraud inference operations.



Figures/memgraph_graph.png

Figure 6.1: Heterogeneous financial transaction graph visualized within the Memgraph Lab environment.

6.5.5 Hybrid HGNN–LSTM Analytical Implementation

The proposed Hybrid HGNN–LSTM analytical module is implemented using PyTorch and PyTorch Geometric to perform graph-based fraud analysis, temporal transaction modeling, and trade metadata learning for intelligent TBML prediction. The implementation receives heterogeneous graph tensors generated from the Memgraph environment, including KYC node features, transactional edge relationships, sequential SWIFT transaction data, and trade deviation attributes before forwarding them into the analytical inference pipeline. The hybrid implementation combines graph attention learning, temporal sequence analysis, and trade feature fusion to improve fraud detection across interconnected financial transaction networks.

The graph branch utilizes a three-layer GATv2Conv architecture with multi-head attention mechanisms to perform neighborhood aggregation and graph message passing across interconnected financial entities. The implemented graph layers process three-dimensional KYC node features including shell-company indicators, jurisdiction information, and dormant-account attributes to generate enriched graph embeddings. **The stacked graph attention layers improve two-hop transactional visibility and enable the model to identify hidden intermediary account relationships and suspicious transaction routing patterns associated with TBML activities.** ReLU activation functions and dropout regularization are integrated after each graph convolution layer to improve representation stability during federated training operations.

The generated graph embeddings are aggregated using `global_mean_pool()` to construct graph-level contextual representations for downstream temporal analysis. The pooled graph context is fused with sequential SWIFT transaction features before being processed through the LSTM branch for temporal anomaly detection and sequential transaction behavior modeling. **The LSTM implementation captures repeated payment structures, abnormal transaction timing behavior, and cyclic financial movement patterns that cannot be identified using standalone graph-learning operations.**

Trade-specific metadata including price deviation and cargo weight-gap attributes are independently processed using multilayer perceptron operations consisting of fully connected layers, Layer Normalization, ReLU activations, and dropout regularization mechanisms. The generated graph embeddings, temporal sequence representations, and

trade feature vectors are fused within the final analytical layer before being forwarded into the dense classification module for fraud prediction. The final classification layer categorizes transactions into predefined classes including CLEAN, WATCHLIST, and FRAUD. Experimental evaluation demonstrated stable federated convergence and high fraud-class Precision–Recall performance across distributed institutional datasets. The overall implementation workflow of the proposed Hybrid HGNN–LSTM analytical module is illustrated in Figure ??.

```

1 # =====
2 # HYBRID HGNN--LSTM IMPLEMENTATION
3 # =====
4
5 --- Graph Attention Layers ---
6 # Multi-hop neighborhood aggregation
7 self.gat1 = GATv2Conv(
8     in_channels=kyc_dim, out_channels=hidden_dim, heads=2, concat=False
9 )
10
11 self.gat2 = GATv2Conv(
12     in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
13 )
14
15 self.gat3 = GATv2Conv(
16     in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
17 )
18
19 --- Temporal Sequence Branch ---
20 # Sequential SWIFT transaction modeling
21 self.lstm = nn.LSTM(
22     input_size=hidden_dim + swift_dim, hidden_size=lstm_hidden, batch_first=True
23 )
24
25 =====
26 # GRAPH MESSAGE PASSING
27 # =====
28
29 x = F.dropout(F.relu(self.gat1(kyc_x, edge_index)), p=0.2, training=self.training)
30
31 x = F.dropout(F.relu(self.gat2(x, edge_index)), p=0.2, training=self.training)
32
33 enriched_nodes = F.dropout(
34     F.relu(self.gat3(x, edge_index)), p=0.2, training=self.training
35 )

```

```

1 # =====
2 # GLOBAL GRAPH POOLING
3 # =====
4
5 graph_context = global_mean_pool(enriched_nodes, batch).unsqueeze(1)
6
7 # =====
8 # GRAPH + TEMPORAL FEATURE FUSION
9 # =====
10
11 graph_context_expanded = graph_context.expand(seq_data.size(0), seq_data.size(1), -1)
12
13 combined_seq = torch.cat([seq_data, graph_context_expanded], dim=-1)
14
15 # Temporal anomaly detection
16 lstm_out, _ = self.lstm(combined_seq)
17
18 # =====
19 # TRADE FEATURE PROCESSING
20 # =====
21
22 trade_emb = self.trade_mlp(trade_features)
23
24 # =====
25 # FINAL FRAUD CLASSIFICATION
26 # =====
27
28 fused_vector = torch.cat([lstm_out[-1], trade_emb], dim=1)
29
30 return self.classifier(fused_vector)

```

Figure 6.2: Implementation workflow of the proposed Hybrid HGNN–LSTM analytical module.

6.5.6 Federated Learning Implementation

The federated learning environment is implemented using the Flower federated learning framework to support distributed privacy-preserving fraud detection across multiple institutional clients. The implementation utilizes a centralized aggregation server and three participating institutional clients, where each client performs localized training of the proposed Hybrid HGNN–LSTM analytical model using institution-specific transaction graphs, sequential SWIFT transaction representations, and fraud classification labels without directly sharing sensitive financial records outside the local analytical environment.

The centralized aggregation server implements a custom federated strategy by extending the `flwr.server.strategy.FedAvg` class and overriding the `aggregate_fit()` operation for distributed parameter aggregation. During each federated communication

round, locally trained model weights generated from participating institutional clients are transmitted to the aggregation server where the **FedAvg** algorithm performs parameter averaging across synchronized client updates to generate a globally optimized fraud detection model. The server initializes the Hybrid HGNN-LSTM analytical architecture using synchronized feature dimensions including three-dimensional KYC node features, one-dimensional SWIFT transaction attributes, two-dimensional trade metadata features, hidden embedding dimensions of size 32, and three-class fraud prediction outputs.

The federated implementation performs ten synchronized communication rounds using complete client participation configurations including `fraction_fit=1.0`, `min_fit_clients=3`, and `min_available_clients=3` to maintain stable distributed convergence across institutional environments. After the final aggregation round, the globally optimized model parameters are converted from Flower parameter streams into NumPy tensors using `parameters_to_ndarrays()` before being serialized and stored as `global_model.npz` for downstream real-time fraud inference operations. **The implemented federated workflow enabled collaborative fraud learning, stable distributed convergence, and privacy-preserving analytical synchronization while maintaining complete institutional data isolation across participating financial organizations.**

6.5.7 User Interface and Compliance Workflow Implementation

The user interface and compliance monitoring environment is implemented using Next.js, TypeScript, Tailwind CSS, Prisma ORM, Zustand state management, and PostgreSQL to support secure institutional fraud monitoring and centralized regulatory investigation workflows. The frontend architecture follows a modular service-oriented implementation consisting of protected dashboard routes, middleware-driven authentication guards, API handlers, Prisma-based database interaction layers, and role-specific monitoring interfaces synchronized with the federated analytical inference environment.

Authentication and Role Validation Pipeline

The authentication workflow is implemented using JWT-based session authorization integrated with centralized middleware validation and protected route interception mechanisms. Incoming requests targeting protected dashboard routes are intercepted through middleware guards where session tokens are validated and mapped against predefined authorization scopes including `BANK_ADMIN` and `REGULATOR`. The validated session context

is subsequently propagated across frontend environments through centralized authentication providers and Zustand-backed session state management before protected dashboard components and compliance services are rendered within the client environment.

Transaction Monitoring and Dashboard Rendering

Fraud predictions generated by the federated Hybrid HGNN–LSTM inference pipeline are synchronized into centralized PostgreSQL audit tables through backend transaction-processing services. Protected Next.js API handlers subsequently retrieve transaction classifications, fraud scores, STR metadata, and institutional monitoring information using Prisma query operations before dynamically rendering analytical outputs within role-specific dashboard interfaces. The banking administrator dashboard supports transaction filtering, fraud investigation workflows, watchlist escalation operations, and false-positive reclassification procedures through synchronized API-driven transaction state updates across centralized audit environments.

STR Generation and Notification Workflow

Confirmed fraudulent transactions trigger Suspicious Transaction Report (STR) generation workflows implemented through secured API endpoints and PostgreSQL-backed audit synchronization services. Institutional administrators initiate STR generation requests through protected dashboard interfaces where transaction metadata, fraud classifications, investigation summaries, institutional remarks, and compliance status information are serialized and persisted into centralized STR audit tables. Generated STR records are subsequently propagated into regulator-facing monitoring environments through synchronized notification services and backend compliance-routing operations.

Regulatory Investigation and Compliance Monitoring

The regulatory monitoring environment consumes institution-generated STR records, fraud-classified transaction histories, compliance metadata, and supporting analytical evidence through protected API routes and Prisma-based database interaction layers. Regulatory authorities can perform investigation workflows including document-verification requests, institutional inquiry procedures, transaction review operations, and account-freeze initiation requests through centralized compliance interfaces synchronized with backend audit environments. The implemented monitoring workflow enabled synchronized fraud prediction, institutional verification, STR lifecycle management, and central-

ized compliance monitoring within a unified real-time TBML analytical platform.

6.6 Summary

This chapter presented the implementation workflow of the proposed TBML detection framework including Kafka-based transaction streaming, heterogeneous graph construction, Hybrid HGNN–LSTM analytical processing, federated collaborative learning, real-time fraud inference, and dashboard-driven compliance monitoring. The chapter discussed the implementation of distributed analytical services, graph-processing pipelines, federated parameter aggregation mechanisms, inference synchronization workflows, and centralized PostgreSQL audit environments operating within the proposed framework.

The implementation chapter further demonstrated how graph attention learning, temporal transaction modeling, trade feature fusion, federated analytical synchronization, and role-based compliance monitoring are integrated within a unified real-time TBML detection environment. The subsequent chapter presents the experimental evaluation, performance analysis, federated convergence results, and monitoring outputs generated by the proposed TBML detection framework.

Chapter 7

Software Testing

CHAPTER 7

SOFTWARE TESTING

This chapter presents the software testing and validation procedures carried out for the proposed TBML detection framework including backend analytical services, distributed transaction-processing pipelines, heterogeneous graph-construction environments, federated analytical synchronization, dashboard-based compliance monitoring, authentication workflows, and institutional investigation operations. The chapter discusses the unit testing, integration testing, and end-to-end system validation workflows performed across the implemented analytical and monitoring environments to ensure reliable execution, synchronization stability, and operational consistency within the proposed framework.

7.1 Testing Environment

The testing process was conducted in a controlled development environment consisting of Python 3.11, PyTorch, PyTorch Geometric, Apache Kafka, Memgraph, PostgreSQL, and Flower federated learning services operating within isolated virtual environments created using `venv`. Frontend monitoring interfaces were implemented and validated using Next.js, TypeScript, Tailwind CSS, Prisma ORM, Zustand state management, and JWT-based authentication workflows.

The testing workflow additionally utilized Playwright-based browser automation for validating protected dashboard routes, institutional transaction workflows, STR generation pipelines, and compliance-monitoring interfaces. Database synchronization latency, transaction propagation consistency, backend API communication, and distributed fraud-inference workflows were validated under controlled institutional transaction loads to ensure reliable execution across distributed analytical environments.

7.2 Unit Testing

Each backend analytical service, dashboard workflow, authentication mechanism, and database synchronization component was independently tested to validate correctness, boundary handling, synchronization stability, and operational consistency before integration into the complete TBML detection framework.

7.2.1 Unit Testing of Authentication and RBAC Module

The authentication and role-based access control environment was independently validated to ensure secure user authentication, authorization management, middleware-driven route protection, credential verification, and institutional role isolation across banking and regulatory dashboard interfaces. The testing workflow additionally verified protected API access, invalid credential handling, unauthorized route restrictions, and role-specific dashboard rendering across distributed monitoring environments.

Table 7.1: Unit Testing of Authentication and RBAC Module

Serial Number of Test Case	1.1
Name of Test Case	Protected Dashboard Access Validation
Feature being Tested	Authentication and Authorization Validation
Description	Validate protected route access, JWT verification, and role-based authorization
Sample Input	Invalid JWT session and unauthorized user request
Expected Output	Unauthorized access denied with authentication error response
Actual Output	Invalid sessions blocked and unauthorized users redirected successfully
Remarks	Successful test

Middleware-driven session validation successfully enforced protected route isolation and role-specific dashboard authorization across institutional monitoring environments. The testing workflow validated JWT token verification, session-expiration handling, credential validation, protected API access control, and middleware-based authorization checks across banking and regulator dashboards. Invalid credentials, unauthorized API requests, expired authentication sessions, and direct access attempts to protected routes were correctly rejected using backend access-control policies before protected resources were rendered. The authentication workflow additionally verified successful role mapping for `BANK_USER` and `ADMIN` entities to ensure secure dashboard isolation and controlled access to institutional fraud-monitoring operations.

7.2.2 Unit Testing of Kafka Transaction Streaming

The Kafka transaction streaming environment was independently tested to validate asynchronous transaction publishing, real-time event synchronization, payload integrity validation, and downstream analytical communication across distributed backend services. The testing workflow additionally verified message delivery consistency, topic-based transaction propagation, malformed payload handling, and reliable synchronization between Kafka producer and consumer services operating within the proposed framework.

Table 7.2: Unit Testing of Kafka Transaction Streaming

Serial Number of Test Case	1.2
Name of Test Case	Publish Valid SWIFT Transaction
Feature being Tested	Kafka Event Streaming and Message Synchronization
Description	Validate transaction publishing and topic-based event synchronization
Sample Input	JSON transaction payload containing sender, receiver, amount, and trade metadata
Expected Output	Transaction published into <code>incoming_swift_messages</code> topic successfully
Actual Output	Transaction streamed and synchronized successfully across Kafka services
Remarks	Successful test

The Kafka streaming module successfully validated asynchronous transaction publishing and backend event synchronization across distributed analytical services. Producer and consumer services maintained reliable message propagation and transaction consistency during continuous event-stream processing operations. Invalid payload structures, malformed transaction metadata, and incomplete financial events were correctly rejected through backend validation workflows before downstream graph-processing and fraud-inference operations were initiated.

7.2.3 Unit Testing of Graph Construction Module

The heterogeneous graph-construction environment was independently tested to validate transactional node generation, relationship mapping, multi-entity graph synchronization, and Cypher-based ingestion workflows within the Memgraph analytical envi-

ronment. The testing workflow additionally verified graph consistency, entity-link generation, transaction propagation accuracy, and downstream graph availability for Hybrid HGNN–LSTM analytical inference operations.

Table 7.3: Unit Testing of Graph Construction Module

Serial Number of Test Case	1.3
Name of Test Case	Heterogeneous Graph Entity Generation
Feature being Tested	Memgraph Graph Construction and Relationship Mapping
Description	Validate transactional node generation and graph relationship synchronization
Sample Input	Valid SWIFT transaction payload with account, trade, and document metadata
Expected Output	Graph nodes and transactional relationships created successfully within Memgraph
Actual Output	Heterogeneous graph entities and multi-hop relationships generated successfully
Remarks	Successful test

The graph-construction workflow successfully generated heterogeneous transactional graphs within the Memgraph environment using Cypher-based ingestion and relationship-mapping operations. Graph entities including `(:Account)`, `(:Transaction)`, and `(:Document)` were correctly synchronized with transactional relationships such as `[ :SENDS ]`, `[ :TO ]`, and `[ :HAS_DOC ]` during continuous event-stream processing. The testing workflow additionally validated graph consistency, entity-link integrity, and multi-hop transactional connectivity required for downstream subgraph extraction and graph-based fraud inference operations within the proposed framework.

7.2.4 Unit Testing of STR Generation Workflow

The STR generation workflow was independently tested to validate suspicious transaction report generation, compliance-record synchronization, fraud-case escalation, and regulator-side notification propagation across distributed institutional monitoring environments. The testing workflow additionally verified report persistence, transaction-metadata preservation, audit consistency, and downstream synchronization between banking dashboards and regulator-facing compliance interfaces.

Table 7.4: Unit Testing of STR Generation Workflow

Serial Number of Test Case	1.4
Name of Test Case	Suspicious Transaction Report Generation
Feature being Tested	STR Generation and Compliance Synchronization
Description	Validate fraud escalation and regulator-side STR synchronization workflow
Sample Input	Fraud-classified transaction selected from institutional monitoring dashboard
Expected Output	STR record generated and synchronized into regulator compliance environment
Actual Output	STR generated, persisted, and synchronized successfully across monitoring services
Remarks	Successful test

The STR workflow successfully validated automated fraud-report generation, PostgreSQL audit synchronization, and regulator-side notification propagation across distributed institutional environments. Generated STR records correctly preserved transaction metadata, fraud classifications, institutional identifiers, and investigation references during downstream compliance synchronization operations. The testing workflow additionally verified successful dashboard synchronization, report persistence, and controlled propagation of suspicious transaction alerts between banking institutions and regulatory monitoring interfaces operating within the proposed framework.

7.2.5 Unit Testing of Notification Workflow

The notification synchronization environment was independently tested to validate fraud-alert propagation, institutional notification delivery, compliance-event synchronization, and regulator-side alert visibility across distributed monitoring interfaces. The testing workflow additionally verified notification persistence, dashboard update consistency, backend event propagation, and synchronization between banking and regulatory compliance environments during fraud escalation operations.

Table 7.5: Unit Testing of Notification Workflow

Serial Number of Test Case	1.5
Name of Test Case	Fraud Alert and Compliance Notification Propagation
Feature being Tested	Notification Synchronization and Alert Delivery
Description	Validate fraud-alert propagation across banking and regulator monitoring environments
Sample Input	Generated STR record linked to fraud-classified transaction
Expected Output	Compliance alerts synchronized into regulator-facing dashboard interfaces
Actual Output	Fraud notifications propagated and synchronized successfully
Remarks	Successful test

The notification workflow successfully validated fraud-alert propagation, institutional investigation updates, and STR lifecycle synchronization across distributed compliance-monitoring environments. Backend event-processing services maintained reliable notification delivery and dashboard synchronization during fraud escalation operations between banking institutions and regulatory authorities. The testing workflow additionally verified notification persistence, regulator-side alert visibility, and consistent compliance-state propagation across protected monitoring interfaces operating within the proposed framework.

7.3 Integration Testing

Integration testing was conducted to validate the interaction, synchronization consistency, and workflow stability between distributed analytical services, graph-processing environments, fraud inference pipelines, dashboard-rendering systems, and regulatory compliance workflows operating within the proposed TBML detection framework.

7.3.1 Integration Testing of Kafka and Memgraph Pipeline

This integration test validated end-to-end synchronization between Kafka-based transaction streaming services and the heterogeneous graph-construction environment operating within Memgraph. The testing workflow additionally verified event-stream con-

sistency, transaction propagation reliability, graph-ingestion stability, and downstream synchronization between distributed backend services.

Table 7.6: Integration Testing of Kafka and Memgraph Pipeline

Serial Number of Test Case	2.1
Name of Test Case	Transaction Stream and Graph Synchronization Workflow
Feature being Tested	Kafka Event Propagation and Memgraph Synchronization
Description	Validate transaction-stream synchronization and heterogeneous graph generation
Sample Input	Valid Kafka transaction payload with trade meta-data and KYC attributes
Expected Output	Transactional graph entities and relationships generated successfully within Memgraph
Actual Output	Kafka events synchronized and heterogeneous graph generated successfully
Remarks	Successful test

The integration workflow successfully validated asynchronous transaction propagation between Kafka producer-consumer services and Memgraph graph-ingestion environments. Streamed SWIFT transaction events were reliably synchronized into graph-processing pipelines without transactional inconsistency or event-loss during continuous backend processing operations. The testing workflow additionally verified stable graph synchronization, multi-entity relationship generation, and downstream availability of transactional subgraphs required for graph-based analytical inference within the proposed framework.

7.3.2 Integration Testing of Fraud Prediction and Dashboard Synchronization

This integration test validated synchronization between fraud-prediction services, PostgreSQL audit environments, and role-based monitoring dashboards operating within the proposed TBML framework. The testing workflow additionally verified database persistence, dashboard synchronization consistency, and real-time rendering of fraud-classified transactions.

Table 7.7: Integration Testing of Fraud Prediction and Dashboard Synchronization

Serial Number of Test Case	2.3
Name of Test Case	Fraud Prediction and Dashboard Rendering Workflow
Feature being Tested	PostgreSQL Synchronization and Dashboard Visualization
Description	Validate fraud-prediction synchronization into institutional monitoring dashboards
Sample Input	Fraud-classified transaction generated from analytical inference pipeline
Expected Output	Fraud transaction persisted and rendered successfully within dashboard interfaces
Actual Output	Dashboard synchronization completed successfully with valid fraud-state rendering
Remarks	Successful test

Fraud-prediction records, transaction metadata, and institutional risk classifications were successfully synchronized into centralized PostgreSQL audit environments before being rendered through protected dashboard interfaces. Dashboard synchronization workflows maintained consistent fraud-state propagation and real-time transaction visibility across distributed institutional monitoring services. The testing workflow additionally validated backend persistence consistency, dashboard update reliability, and secure rendering of fraud-monitoring data across role-specific compliance environments.

7.3.3 Integration Testing of STR Generation Workflow

This integration test validated the complete workflow between institutional fraud verification, STR generation services, regulator-side synchronization, and compliance-notification propagation operating within distributed monitoring environments. The testing workflow additionally verified fraud escalation consistency, report persistence, and synchronized regulator visibility across institutional compliance pipelines.

The STR synchronization workflow successfully validated fraud escalation, compliance-report generation, regulator-side dashboard synchronization, and distributed notification propagation across institutional monitoring environments. Generated STR records consistently preserved fraud classifications, investigation metadata, institutional identifiers, and audit references during downstream compliance-processing operations. The testing

workflow additionally verified successful propagation of compliance alerts and synchronized visibility of suspicious transaction reports across banking and regulatory monitoring interfaces.

Table 7.8: Integration Testing of STR Generation Workflow

Serial Number of Test Case	2.4
Name of Test Case	Fraud Escalation and STR Compliance Workflow
Feature being Tested	STR Lifecycle Management and Compliance Synchronization
Description	Validate fraud escalation and regulator-side STR synchronization workflow
Sample Input	Fraud transaction escalated by institutional administrator
Expected Output	STR generated and synchronized successfully within regulator compliance dashboard
Actual Output	Fraud escalation and STR synchronization completed successfully
Remarks	Successful test

7.3.4 Integration Testing of Authentication and Protected Dashboard Rendering

This integration test validated synchronization between JWT-based authentication services, middleware-driven authorization guards, protected API routes, and role-specific dashboard-rendering environments operating within the proposed framework. The testing workflow additionally verified secure session propagation, authorization consistency, and protected resource accessibility across institutional monitoring interfaces.

Middleware-driven authentication workflows successfully validated session authorization, protected route synchronization, secure API accessibility, and institutional role isolation across distributed monitoring interfaces. Dashboard-rendering operations correctly enforced role-specific access-control policies for `BANK_USER`, `ADMIN`, and `REGULATOR` entities during backend synchronization workflows. The testing workflow additionally verified invalid-session rejection, unauthorized route restrictions, and secure propagation of authenticated dashboard resources across institutional compliance-monitoring environments.

Table 7.9: Integration Testing of Authentication and Protected Dashboard Rendering

Serial Number of Test Case	2.5
Name of Test Case	Authenticated Dashboard Access and Rendering Workflow
Feature being Tested	Session Validation and Role-Based Dashboard Authorization
Description	Validate authenticated dashboard rendering and protected route authorization
Sample Input	Valid JWT-authenticated institutional user session
Expected Output	Authorized dashboard rendered successfully based on institutional role
Actual Output	Protected dashboard rendered successfully with valid role authorization
Remarks	Successful test

7.4 System Testing

System testing was conducted to validate the operational stability, concurrent transaction-handling capability, database synchronization performance, and workflow-level response consistency of the proposed TBML detection framework under distributed institutional monitoring environments. The testing workflow focused on browser-based end-to-end validation, protected dashboard execution, concurrent PostgreSQL scalability analysis, API response-time benchmarking, and operational latency evaluation across fraud-monitoring and compliance-management services.

7.4.1 Browser Automation and Workflow Validation

Browser-based end-to-end validation was performed using Playwright automation workflows executed within Chromium testing environments. The testing workflow validated protected dashboard rendering, role-based authentication, STR lifecycle execution, fraud-notification propagation, compliance-action synchronization, and institutional transaction visibility restrictions across distributed frontend and backend services. All automated workflow executions completed successfully without unauthorized access, synchronization failure, or dashboard inconsistency during operational validation.

Table 7.10: Browser Automation and Workflow Validation Results

Workflow Validation	Status
Admin Authentication and Dashboard Rendering	Successful
Fraud Case Retrieval and STR Visibility	Successful
GET_DOCUMENTS Compliance Workflow	Successful
FREEZE_ACCOUNT Regulatory Workflow	Successful
Fraud Verdict Marking Workflow	Successful
Bank User STR Generation Workflow	Successful
Fraud Notification Propagation Workflow	Successful
Protected Route and RBAC Validation	Successful
Institutional Transaction Visibility Restriction	Successful
Sign-In Page Smoke Testing	Successful

The Playwright automation workflow successfully validated sequential execution of institutional fraud-monitoring operations across protected dashboard environments. Browser automation additionally verified secure session persistence, API synchronization consistency, workflow-level authorization enforcement, and stable dashboard rendering across **ADMIN** and **BANK_USER** operational environments. The testing workflow further confirmed successful STR generation, fraud escalation, regulator-side compliance synchronization, and distributed notification propagation across automated end-to-end execution workflows.

7.4.2 Concurrent Database Scalability and API Performance Analysis

Concurrent database scalability testing was conducted to evaluate PostgreSQL synchronization stability, dashboard query performance, fraud-filtering latency, and STR retrieval response consistency under increasing institutional user loads. The benchmarking workflow was implemented using Prisma ORM and Next.js backend services, where each simulated institutional user executed five continuous transactional requests during concurrent dashboard-access operations. Controlled concurrency execution was

implemented using batched asynchronous request propagation with pooled PostgreSQL database connections and a maximum limit of 15 active concurrent connections to regulate transactional synchronization, prevent backend resource exhaustion, and maintain stable query-execution behavior during high-volume monitoring workloads.

Latency benchmarking was performed by capturing request-initiation and response-completion timestamps during concurrent API execution workflows. For each transactional request, the benchmarking workflow first recorded the request initiation timestamp using `startTime = performance.now()`, followed by asynchronous API request execution and backend-response synchronization. Upon successful completion of the transactional request, the response-completion timestamp was recorded using `endTime = performance.now()`, and the operational latency was computed using `latency = endTime - startTime`. Concurrent workload execution was implemented using parallel asynchronous request propagation, where multiple simulated institutional users executed simultaneous dashboard and compliance-query operations under controlled concurrency constraints.

Average latency measurements represented the mean response time observed across all successful transactional requests, while P95 latency measurements captured upper-bound response behavior experienced by 95% of requests during peak synchronization conditions. The benchmarking workflow additionally evaluated request throughput, API synchronization consistency, transaction persistence reliability, and stable PostgreSQL query execution across distributed compliance-monitoring environments.

The benchmarking metrics were computed using the following operational latency equations:

$$L_{avg} = \frac{1}{N} \sum_{i=1}^N (T_{response_i} - T_{request_i}) \quad (7.1)$$

$$L_{95} = \text{Percentile}_{95}(L_1, L_2, \dots, L_N) \quad (7.2)$$

Table 7.11: Dashboard Transaction Retrieval Scalability Analysis

Concurrent Users	Total Re-quests	Avg Latency (ms)	P95 Latency (ms)	Success Rate
10	50	297.33	884.28	100%
50	250	156.05	174.75	100%
100	500	159.99	181.30	100%
250	1250	158.53	176.99	100%
500	2500	162.14	180.72	100%

The dashboard transaction retrieval workflow demonstrated stable query scalability and consistent dashboard-rendering performance across increasing institutional user loads. Average latency values remained nearly constant between 50 and 500 concurrent users, indicating efficient PostgreSQL query execution, optimized transactional indexing, and stable backend synchronization during high-frequency dashboard-access operations. The relatively low variation between average and P95 latency measurements further indicated predictable response behavior and controlled query-execution overhead under concurrent transactional workloads.

Table 7.12: Fraud Transaction Filtering Scalability Analysis

Concurrent Users	Total Re-quests	Avg Latency (ms)	P95 Latency (ms)	Success Rate
10	50	159.58	174.04	100%
50	250	158.15	181.58	100%
100	500	147.56	170.37	100%
250	1250	148.18	164.58	100%
500	2500	263.80	731.88	100%

The fraud-transaction filtering workflow demonstrated efficient analytical-query execution and stable institution-specific transaction retrieval during concurrent dashboard operations. Lower average latency values under moderate workloads indicated optimized fraud-state filtering and indexed transactional query execution across backend analytical services. Increased P95 latency under 500-user workloads indicated temporary synchronization overhead during peak concurrent analytical filtering operations; however, the framework maintained complete request reliability, stable API synchronization, and consistent fraud-state propagation throughout distributed monitoring workflows.

Table 7.13: STR Compliance Report Retrieval Scalability Analysis

Concurrent Users	Total Requests	Avg Latency (ms)	P95 Latency (ms)	Success Rate
10	50	2694.21	7311.50	100%
50	250	2101.61	4791.52	100%
100	500	856.39	2149.27	100%
250	1250	467.03	600.67	100%
500	2500	465.62	596.76	100%

The STR compliance retrieval workflow exhibited comparatively higher response latency due to computationally intensive compliance-report aggregation, multi-table relational joins, transaction-history synchronization, and regulator-side metadata retrieval operations executed during report-generation workflows. The progressive reduction in average latency as concurrent workloads increased indicated improved query-plan stabilization, optimized relational-query execution, and more efficient pooled database synchronization during sustained transactional processing. Additionally, repeated execution of compliance-retrieval operations enabled PostgreSQL shared-buffer and query-cache utilization for frequently accessed transactional rows and relational metadata, thereby reducing disk-access overhead and improving subsequent query-response performance under continuous concurrent workloads. Despite the increased computational overhead associated with compliance-report aggregation and relational join operations, the framework maintained stable request consistency and complete request success rates across distributed regulatory monitoring environments.

The conducted testing workflows validated the synchronization reliability, operational stability, and execution consistency of the proposed TBML detection framework across distributed monitoring environments. The testing results confirmed stable backend communication, reliable transactional synchronization, controlled API response behavior, and successful workflow execution across graph-processing, federated analytical inference, dashboard synchronization, and compliance-management services. Concurrent scalability validation additionally demonstrated stable PostgreSQL query execution and controlled latency characteristics under increasing institutional workloads.

7.5 Model Performance Validation

Chapter 8

Results & Discussions

CHAPTER 8

RESULTS & DISCUSSIONS

Every chapter should start with an introduction paragraph. This paragraph should brief about the flow of the chapter. This introduction can be limited within 4 to 5 sentences. The chapter heading should be appropriately modified (a sample heading is shown for this chapter). But don't start the introduction paragraph in the chapters like "This chapter deals with...". Instead you should bring in the highlights of the chapter in the introduction paragraph.

8.1 Contents of this chapter

All the results obtained for your objectives should be discussed in this chapter. This chapter should contain the following sections as per the project.

1. Simulation results
2. Experimental results
3. Performance Comparison
4. Inferences drawn from the results obtained

All the figures should be properly explained by bringing the scenarios of the design done in the project. A detailed discussion of results obtained should be done in this chapter.

8.2 Tables in thesis

- All Table Caption should be in Sentence Case, TNR 10 Pt. It should be of the Format:
 - Table 1.1 Results of the experiment(Centered)
- It should be cited as Table 1.1.
- Caption should appear above the Table.
- Table Header and the entries should be of Font TNR 10 Pt, Justified.
- For wider Table, the page orientation can be Landscape.

- For Larger Table, it can run to pages and the header should be repeated for each page of the Table.
- Table must be adjusted to fit in the page and no single row is left out for a new page.

Sample Table ?? is given below for your reference,

Table 8.1: Country List

Country Name or Area Name	ISO ALPHA 2 Code	ISO ALPHA 3 Code	ISO numeric Code
Afghanistan	AF	AFG	004
Aland Islands	AX	ALA	248
Albania	AL	ALB	008
Algeria	DZ	DZA	012
American Samoa	AS	ASM	016
Andorra	AD	AND	020
Angola	AO	AGO	024

8.3 Math equation in thesis

All equation should be written using equation editor or using an equivalent tool.

- Equations should be numbered as : 1.1, 1.2 ...
- Equation should be Centered, 12 Pt, TNR.
- Equation number should be right Justified
- It should be cited as Eqn. 1.1.
- If the sentence starts by citing an equation, then it should be written as Equation 1.1 For example, Equation 5.1 states the Pythagoras theorem.

For example in Eqn. ??, The well known Pythagorean theorem $x^2 + y^2 = z^2$ was proved to be invalid for other exponents. Meaning the next equation has no integer solutions:

$$x^n + y^n = z^n \quad (8.1)$$

The mass-energy equivalence is described by the famous equation in Eqn. ??

$$E = mc^2 \quad (8.2)$$

discovered in 1905 by Albert Einstein.

8.4 Interpretation of results

The chapters should not end with figures, instead bring the paragraph explaining about the figure at the end followed by a summary paragraph.

After elaborating the various sections of the chapter (From Chapter 2 onwards), a summary paragraph should be written discussing the highlights of that particular chapter. This summary paragraph should not be numbered separately. This paragraph should connect the present chapter to the next chapter.

Chapter 9

Conclusion and Future Scope

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

Every chapter should start with an introduction paragraph. This paragraph should be brief about the flow of the chapter. This introduction can be limited within 4 to 5 sentences. The chapter heading should be appropriately modified (a sample heading is shown for this chapter). But don't start the introduction paragraph in the chapters like "This chapter deals with...". Instead you should bring in the highlights of the chapter in the introduction paragraph.

9.1 Conclusion

This chapter should not contain an introduction paragraph like other chapters. You can directly write conclusion of the work done under this section. Typically this section can have 3 to 4 paragraphs.

First paragraph should bring in the scenario of the project and every objective should be explained here.

Second paragraph should say how the objectives are implemented and achieved.

Last paragraph should draw the conclusions from each objective with quantitative results, performance improvement etc.

9.2 Future Scope

Briefly discuss the constraints and limitations of the project and state the possibilities of extending the work in future.

9.3 Learning Outcomes of the Project

- List the learning outcomes here
- List a minimum of 5 learning outcomes

Appendix A

Code

APPENDIX A

CODE

A.1 First Appendix

You can use `tcbllisting` for creating the code snippets. The following example illustrates how one can customize the `tcbllisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

- [1] IMTF Compliance Technologies, *When trade becomes a cover: Unpacking trade-based money laundering*, 2024. [Online]. Available: <https://imtf.com/blog/when-trade-becomes-a-cover-unpacking-trade-based-money-laundering/>
- [2] Financial Action Task Force and Egmont Group, “Trade-based money laundering: Trends and developments,” FATF, Paris, France, Tech. Rep., 2020. [Online]. Available: <https://fatf-gafi.org>
- [3] United Nations Office on Drugs and Crime, *Money laundering and Illicit financial flows overview*, 2024. [Online]. Available: <https://unodc.org>
- [4] Europol, *Criminal finances, money laundering and asset recovery*, 2024. [Online]. Available: <https://europa.eu>
- [5] J. Saenz and J. Lewer, “Quantifying illicit financial flows and trade misinvoicing in developing economies,” *Journal of Financial Crime Research*, vol. 18, no. 2, pp. 112–128, 2024.
- [6] R. Z. Z. L. W. Y. D. N. K.-Y. L. Jiani Fan Lwin Khin Shar, “Deep learning approaches for anti-money laundering on mobile transactions: Review, framework, and directions,” *IEEE Internet of Things Journal*, vol. 13, pp. 10 407–10 428, 2026, ISSN: 2327-4662. DOI: 10.1109/JIOT.2026.3653434
- [7] A. I. Rasmus Ingemann Tuffveson Jensen, “Fighting money laundering with statistics and machine learning,” *IEEE Access*, vol. 11, pp. 8889–8903, 2023, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2023.3239549
- [8] J. P. Y. G. G. Y. Jie Song Sijia Zhang, “Dimension expansion for learning money laundering activities hidden in transaction network,” *IEEE Transactions on Computational Social Systems*, vol. 13, pp. 251–262, 2026, ISSN: 2329-924X. DOI: 10.1109/TCSS.2025.3599534
- [9] P. Z. J. P. Y. G. G. Y. Jie Song Sijia Zhang, “Illicit social accounts? anti-money laundering for transactional blockchains,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 391–404, 2025, ISSN: 1556-6021. DOI: 10.1109/TIFS.2024.3518068

- [10] C. J. Zhong Li Xueting Yang, “Multi-view graph-based hierarchical representation learning for money laundering group detection,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 2035–2050, 2025, ISSN: 1556-6021. DOI: 10.1109/TIFS.2025.3529321
- [11] E. S. Abdul Haluk Batur Gezici, “Pagerank-based unsupervised deep vertex representations for anti-money laundering detection,” *IEEE Access*, vol. 13, pp. 196 935–196 950, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3634197
- [12] S. A. C. P. D. P. A. M. Md. Rezaul Karim Felix Hermsen, “Scalable semi-supervised graph learning techniques for anti money laundering,” *IEEE Access*, vol. 12, pp. 50 012–50 029, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3383784
- [13] B. Y. Kyungmo Koo Minyoung Park, “A suspicious financial transaction detection model using autoencoder and risk-based approach,” *IEEE Access*, vol. 12, pp. 68 926–68 939, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3399824